

DOCKER CHALLENGES

1.Create a customized Docker image using a Dockerfile.

Create index.html file

```
root@ip-172-31-26-239: /home/ubuntu
<!DOCTYPE html>
<html>
<head>
  <title>Docker Challenge</title>
</head>
<body>
  <h1>Hello from my customized Docker container!!</h1>
  <h2>welcome to Nginx from Dockerfile</h2>
</body>
</html>
~
~
~
~
```

Create Dockerfile

```
root@ip-172-31-26-239: /home/ubuntu
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
|
```

docker build -t my-custom-nginx .

```
root@ip-172-31-26-239:/home/ubuntu# docker build -t my-custom-nginx .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 63.37MB
Step 1/3 : FROM nginx:latest
----> 5aca99593157
Step 2/3 : COPY index.html /usr/share/nginx/html/index.html
----> 2f13ca5a4ff9
Step 3/3 : EXPOSE 80
----> Running in dd6292f3be42
----> Removed intermediate container dd6292f3be42
----> 96fa51fbcc00
Successfully built 96fa51fbcc00
Successfully tagged my-custom-nginx:latest
root@ip-172-31-26-239:/home/ubuntu# |
```

→Run Container

docker run -d -p 8080:80 --name nginx-container

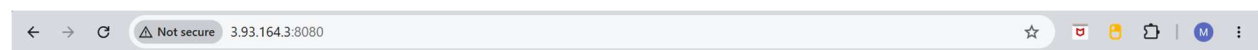
my-custom-nginx

docker ps

curl localhost:8080

```
root@ip-172-31-26-239:/home/ubuntu# docker run -d -p 8080:80 --name nginx-container my-custom-nginx
11a2af8f506da567bf3e8fa34ae9d1fee9a12785fa606764fe1a8d7fa7fc8224
root@ip-172-31-26-239:/home/ubuntu# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
11a2af8f506d   my-custom-nginx  "/docker-entrypoint..." 6 seconds ago  Up 6 seconds  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
b69d55e858b4   jenkins/jenkins:lts  "/usr/bin/tini -- /u..." 2 hours ago    Up 2 hours    0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp, 0.0.0.0:9090->8080/tcp, [::]:9090->8080/tcp
root@ip-172-31-26-239:/home/ubuntu# curl localhost:8080
<!DOCTYPE html>
<html>
<head>
  <title>Docker Challenge</title>
</head>
<body>
  <h1>Hello from my customized Docker container!!</h1>
  <h2>welcome to Nginx from Dockerfile</h2>
</body>
</html>
root@ip-172-31-26-239:/home/ubuntu#
```

Verify in browser



Hello from my customized Docker container!!

welcome to Nginx from Dockerfile

2. Push the image to Docker Hub.

Login to Docker Hub

docker login

```
root@ip-172-31-26-239:/home/ubuntu# docker login
Authenticating with existing credentials... [Username: manisha3299]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
root@ip-172-31-26-239:/home/ubuntu#
```

→ Tag the image

```

root@ip-172-31-26-239:/home/ubuntu# docker tag my-custom-nginx:latest manisha3299/my-custom-nginx:latest
root@ip-172-31-26-239:/home/ubuntu# docker images

```

IMAGE	ID	DISK USAGE	CONTENT SIZE	In Use
142083399660.dkr.ecr.us-east-1.amazonaws.com/nginx-custom:v1	2ad06826471f	584MB	195MB	
142083399660.dkr.ecr.us-east-1.amazonaws.com/nginx-scan-test:latest	d679e3e7dc30	238MB	63.1MB	
alpine:latest	5b10f432ef3d	13.1MB	3.95MB	
amazoncorretto:17	823513044539	711MB	221MB	
amazonlinux:latest	ad882807c6c8	227MB	59.3MB	
jenkins/jenkins:lts	01c992ffef29	799MB	289MB	U
manisha3299/my-custom-nginx:latest	96fa51fbcc00	238MB	63.1MB	U
manisha3299/nginx-custom:v1	2ad06826471f	584MB	195MB	
manisha3299/nginx-custom:v2	d679e3e7dc30	238MB	63.1MB	
manisha3299/nginx-custom:v3	6f62f7e1699a	742MB	233MB	
maven:3.9-amazoncorretto-17	5417f4bccfba	1.51GB	442MB	
multi-stage-app:v1	49b64aff6755	742MB	233MB	
my-custom-nginx:latest	96fa51fbcc00	238MB	63.1MB	U
nginx-custom:v1	2ad06826471f	584MB	195MB	
nginx-custom:v2	d679e3e7dc30	238MB	63.1MB	
nginx:latest	5aca99593157	241MB	66MB	
single-stage-app:v1	2ff10cb2229c	1.65GB	496MB	U
sonarqube:latest	4671314254b0	2.53GB	1.08GB	
tomcat-alpine:v1	33e90ff563a2	296MB	82.7MB	
tomcat:latest	f8d2287df1bb	589MB	158MB	
ubuntu:latest	f3d28607ddd7	160MB	45.3MB	

```

root@ip-172-31-26-239:/home/ubuntu#

```

→ Push to Docker Hub

docker push manisha3299/my-custom-nginx:latest

```

root@ip-172-31-26-239:/home/ubuntu# docker push manisha3299/my-custom-nginx:latest
The push refers to repository [docker.io/manisha3299/my-custom-nginx]
45381ecb0e2f: Mounted from manisha3299/nginx-custom
13fd728be9eb: Mounted from manisha3299/nginx-custom
77057773781e: Pushed
5b4d6ff92fc4: Mounted from manisha3299/nginx-custom
830625e1ac85: Mounted from manisha3299/nginx-custom
5431d0092ffd: Mounted from manisha3299/nginx-custom
b4a248c845e5: Mounted from manisha3299/nginx-custom
7f8b1a2b17d8: Mounted from manisha3299/nginx-custom
latest: digest: sha256:96fa51fbcc00fbc0548bf3e87728307736ccad802fbc31d6c60bf7ea358a1bcc size: 2043
root@ip-172-31-26-239:/home/ubuntu#

```

→ verified in Docker Hub repository page

The screenshot shows the Docker Hub interface for the repository `manisha3299/my-custom-nginx`. The page is titled "Repositories" and lists all repositories within the `manisha3299` namespace. The repository `manisha3299/my-custom-nginx` is listed with a "Last Pushed" time of "1 minute ago", a "Contains" type of "IMAGE", and a "Visibility" of "Public".

Name	Last Pushed	Contains	Visibility
manisha3299/my-custom-nginx	1 minute ago	IMAGE	Public

The screenshot shows the Docker Hub interface for a repository named 'manisha3299/my-custom-nginx'. The left sidebar contains navigation links: Repositories, Hardened Images, Collaborations, Settings, Default privacy, Notifications, Billing, Usage, Pulls, and Storage. The main content area shows the repository details, including the name 'manisha3299/my-custom-nginx', the last push time '1 minute ago', and a star count of 0. Below this, there are tabs for General, Tags, Image Management, Collaborators, Webhooks, and Settings. The 'Tags' tab is active, showing a table with one tag: 'latest'. The table columns are Tag, OS, Type, Pulled, and Pushed. The 'latest' tag is of type 'Image', pulled 'less than 1 day' ago, and pushed '1 minute' ago. On the right, there are sections for 'Docker commands' with a command 'docker push manisha3299 custom-nginx:tagname' and a 'buildcloud' logo with text 'Build with Docker Build Cloud'.

3. Push the same image to Amazon ECR.

→ Verify AWS CLI Configuration

aws sts get-caller-identity

→ Create ECR Repository

aws ecr create-repository --repository-name my-custom-nginx

```
root@ip-172-31-26-239:/home/ubuntu# aws sts get-caller-identity
{
  "UserId": "142083399660",
  "Account": "142083399660",
  "Arn": "arn:aws:iam::142083399660:root"
}
root@ip-172-31-26-239:/home/ubuntu# aws ecr create-repository --repository-name my-custom-nginx
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:us-east-1:142083399660:repository/my-custom-nginx",
    "registryId": "142083399660",
    "repositoryName": "my-custom-nginx",
    "repositoryUri": "142083399660.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx",
    "createdAt": "2026-06-05T13:46:24.987000+00:00",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": {
      "scanOnPush": false
    },
    "encryptionConfiguration": {
      "encryptionType": "AES256"
    }
  }
}
```

→ Login Docker to ECR

aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 142083399660.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx

```
root@ip-172-31-26-239:/home/ubuntu# aws ecr get-login-password --region us-east-1 |
docker login --username AWS --password-stdin 142083399660.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx

WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
root@ip-172-31-26-239:/home/ubuntu#
```

→ Tag the Image

docker tag my-custom-nginx:latest 142083399660.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx:latest

```
root@ip-172-31-26-239:/home/ubuntu# docker tag my-custom-nginx:latest 142083399660.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx:latest
root@ip-172-31-26-239:/home/ubuntu# docker images
```

IMAGE	ID	DISK USAGE	Info →	U	In Use
142083399660.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx:latest	96fa51fbcc00	238MB	63.1MB	U	
142083399660.dkr.ecr.us-east-1.amazonaws.com/nginx-custom:v1	2ad06826471f	584MB	195MB		
142083399660.dkr.ecr.us-east-1.amazonaws.com/nginx-scan-test:latest	d679e3e7dc30	238MB	63.1MB		
alpine:latest	5b10f432ef3d	13.1MB	3.95MB		
amazoncorretto:17	823513044539	711MB	221MB		

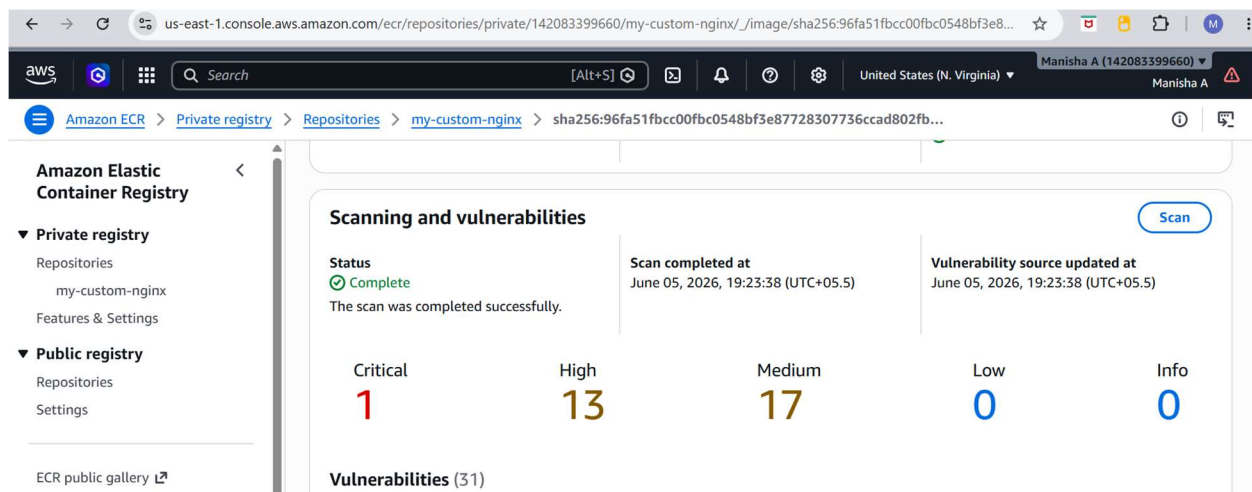
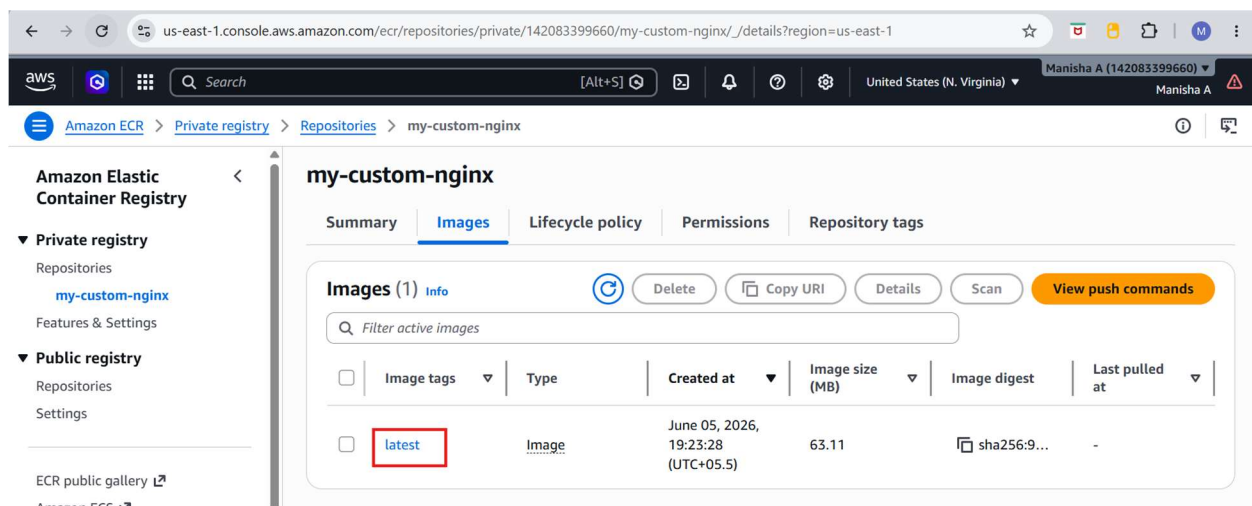
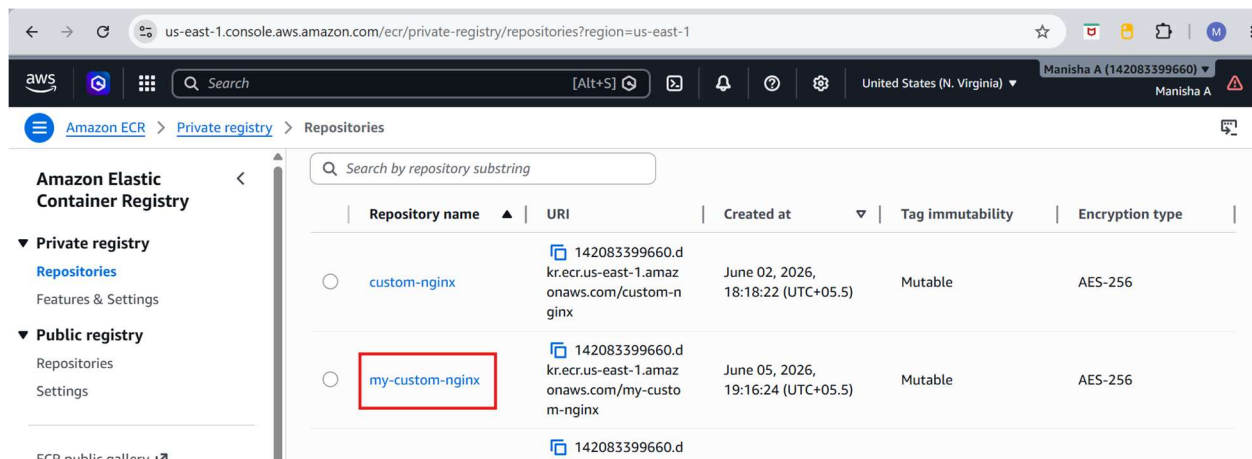
→ Push Image to ECR

docker push 142083399660.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx:latest

```
root@ip-172-31-26-239:/home/ubuntu# docker push 142083399660.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx:latest
The push refers to repository [142083399660.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx]
830625e1ac85: Pushed
b4a248c845e5: Pushed
7f8b1a2b17d8: Pushed
13fd728be9eb: Pushed
45381ecb0e2f: Pushed
77057773781e: Pushed
5431d0092ffd: Pushed
5b4d6ff92fc4: Pushed
latest: digest: sha256:96fa51fbcc00fbc0548bf3e87728307736ccad802fbc31d6c60bf7ea358a1bcc size: 2043
root@ip-172-31-26-239:/home/ubuntu#
```

Verify in AWS Console

→ AWS Console → ECR → Repositories → my-custom-nginx → Images



4.Provision one EC2 instance using Terraform and install Jenkins.

Install with the below command

```
wget -O - https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
```

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(grep -oP '(?<=UBUNTU_CODENAME=).*' /etc/os-release || lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list
```

```
root@ip-172-31-26-239: /home/ubuntu
root@ip-172-31-26-239:/home/ubuntu# wget -O - https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(grep -oP '(?<=UBUNTU_CODENAME=).*' /etc/os-release || lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list
--2026-06-06 09:50:55-- https://apt.releases.hashicorp.com/gpg
Resolving apt.releases.hashicorp.com (apt.releases.hashicorp.com)... 18.160.10.45, 18.160.10.71, 18.160.10.126, ...
Connecting to apt.releases.hashicorp.com (apt.releases.hashicorp.com)|18.160.10.45|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3980 (3.9K) [binary/octet-stream]
Saving to: 'STDOUT'

-          100%[=====>]    3.89K  --.-KB/s    in 0s

2026-06-06 09:50:55 (454 MB/s) - written to stdout [3980/3980]

deb [arch=amd64 signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com resolute main
```

sudo apt update && sudo apt install terraform

```
root@ip-172-31-26-239: /home/ubuntu
root@ip-172-31-26-239:/home/ubuntu# sudo apt update && sudo apt install terraform
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease
Get:4 https://apt.releases.hashicorp.com resolute InRelease [11.2 kB]
Get:5 https://apt.releases.hashicorp.com resolute/main amd64 Packages [238 kB]
Hit:6 http://security.ubuntu.com/ubuntu resolute-security InRelease
Fetched 250 kB in 0s (1168 kB/s)
39 packages can be upgraded. Run 'apt list --upgradable' to see them.
Installing:
  terraform

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 39
  Download size: 34.8 MB
  Space needed: 117 MB / 8608 MB available

Get:1 https://apt.releases.hashicorp.com resolute/main amd64 terraform amd64 1.5.5-1 [34.8 MB]
Fetched 34.8 MB in 0s (202 MB/s)
Selecting previously unselected package terraform.
(Reading database ... 95441 files and directories currently installed.)
```

Verify whether the terraform is installed

Terraform --version

```
root@ip-172-31-26-239:/home/ubuntu# terraform --version
Terraform v1.15.5
on linux_amd64
root@ip-172-31-26-239:/home/ubuntu# aws|
```

Configure aws

aws configure

aws sts get-caller-identity

```
root@ip-172-31-26-239:/home/ubuntu# aws configure
AWS Access Key ID [*****LYUR]:
AWS Secret Access Key [*****wr6T]:
Default region name [us-east-1]:
Default output format [json]:
root@ip-172-31-26-239:/home/ubuntu# aws sts get-caller-identity
{
  "UserId": "142083399660",
  "Account": "142083399660",
  "Arn": "arn:aws:iam::142083399660:root"
}
root@ip-172-31-26-239:/home/ubuntu# |
```

Now create a directory

Mkdir terraform-jenkins

Navigate to that directory

Cd terraform-jenkins

```
root@ip-172-31-26-239:/home/ubuntu# mkdir terraform-jenkins
root@ip-172-31-26-239:/home/ubuntu# cd terraform-jenkins/
root@ip-172-31-26-239:/home/ubuntu/terraform-jenkins# |
```

Lets create some files inside the directory

Touch main.tf variables.tf terraform.tfvars userdata.sh

```
root@ip-172-31-26-239:/home/ubuntu/terraform-jenkins# touch main.tf variables.tf
terraform.tfvars userdata.sh
root@ip-172-31-26-239:/home/ubuntu/terraform-jenkins# ls
main.tf  terraform.tf  variables.tf  varsuserdata.sh
root@ip-172-31-26-239:/home/ubuntu/terraform-jenkins# |
```

Add User Data for Jenkins Installation

→ Add a User Data script to the EC2 instance resource. User Data allows AWS to execute commands automatically when the EC2 instance launches for the first time.

→ This approach automates the installation and configuration of Jenkins, eliminating the need to manually connect to the server and perform installation steps

→ Create user data script

```
#!/bin/bash
```

```
# Update packages
```

```
apt update -y
```

```
# Install Java 21
```

```
apt install openjdk-21-jdk -y
```

```
# Create keyrings directory
```

```
mkdir -p /etc/apt/keyrings
```

```
# Add Jenkins key
```

```
wget -O /etc/apt/keyrings/jenkins-keyring.asc \
```

```
https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
```

```
# Add Jenkins repository
```

```
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] \
```

```
https://pkg.jenkins.io/debian-stable binary/" | sudo tee \
```

```
/etc/apt/sources.list.d/jenkins.list > /dev/null
```

```
# Update packages again
```

```
apt update -y
```

```
# Install Jenkins
```

```
apt install jenkins -y
```

```
# Enable and start Jenkins
```

```
systemctl enable jenkins
```

```
systemctl start jenkins
```

```
root@ip-172-31-26-239: /home/ubuntu/terraform-jenkins
#!/bin/bash
# Update packages
apt update -y
# Install Java 21
apt install openjdk-21-jdk -y
# Create keyrings directory
mkdir -p /etc/apt/keyrings
# Add Jenkins key
wget -O /etc/apt/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
# Add Jenkins repository
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] \
https://pkg.jenkins.io/debian-stable binary/" | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null
# Update packages again
apt update -y
# Install Jenkins
apt install jenkins -y
# Enable and start Jenkins
systemctl enable jenkins
systemctl start jenkins
```

→ add variables in Create **vi** variables.tf

```
root@ip-172-31-26-239: /home/ubuntu/terraform-jenkins
variable "aws_region" {
  default = "us-east-1"
}

variable "instance_type" {
  default = "t2.small"
}

variable "key_name" {
  description = "AWS Key Pair Name"
}
```

Create terraform.tfvars using vi

```
root@ip-172-31-26-239:/home/ubuntu/terraform-jenkins# ls
main.tf  terraform.tf  variables.tf  varsuserdata.sh
root@ip-172-31-26-239:/home/ubuntu/terraform-jenkins# vi userdata.sh
root@ip-172-31-26-239:/home/ubuntu/terraform-jenkins# vi variables.tf
root@ip-172-31-26-239:/home/ubuntu/terraform-jenkins# vi terraform.tfvars

root@ip-172-31-26-239:/home/ubuntu/terraform-jenkins# cat terraform.tfvars
key_name = "jen"
root@ip-172-31-26-239:/home/ubuntu/terraform-jenkins#
```

Create main.tf

provider "aws" {

```
    region = var.aws_region
}
```

```
# Get latest Ubuntu AMI
```

```
data "aws_ami" "ubuntu" {
    most_recent = true
```

```
    owners = ["099720109477"]
```

```
    filter {
```

```
        name = "name"
```

```
        values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
```

```
    }
```

```
}
```

```
# Security Group
```

```
resource "aws_security_group" "jenkins_sg" {
```

```
    name = "jenkins-security-group"
```

```
    ingress {
```

```
        description = "SSH"
```

```
        from_port = 22
```

```
        to_port = 22
```

```
        protocol = "tcp"
```

```
        cidr_blocks = ["0.0.0.0/0"]
```

```
    }
```

```
ingress {  
    description = "Jenkins"  
    from_port   = 8080  
    to_port     = 8080  
    protocol    = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
}
```

```
ingress {  
    description = "HTTP"  
    from_port   = 80  
    to_port     = 80  
    protocol    = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
}
```

```
egress {  
    from_port = 0  
    to_port   = 0  
    protocol  = "-1"  
    cidr_blocks = ["0.0.0.0/0"]  
}  
}
```

EC2 Instance


```
resource "aws_instance" "jenkins_server" {  
  ami           = data.aws_ami.ubuntu.id  
  instance_type = var.instance_type  
  key_name      = var.key_name  
  vpc_security_group_ids = [aws_security_group.jenkins_sg.id]  
  
  user_data = file("userdata.sh")  
  
  tags = {  
    Name = "Jenkins-Server"  
  }  
}  
  
# Output Public IP  
output "public_ip" {  
  value = aws_instance.jenkins_server.public_ip  
}
```

root@ip-172-31-26-239: /home/ubuntu/terraform-jenkins

```
provider "aws" {
  region = var.aws_region
}

# Get latest Ubuntu AMI
data "aws_ami" "ubuntu" {
  most_recent = true

  owners = ["099720109477"]

  filter {
    name = "name"
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
  }
}

# Security Group
resource "aws_security_group" "jenkins_sg" {
  name = "jenkins-security-group"

  ingress {
    description = "SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "Jenkins"
    from_port   = 8080
    to_port     = 8080
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port   = 0
  }
}
```

root@ip-172-31-26-239: /home/ubuntu/terraform-jenkins

```
    from_port = 8080
    to_port   = 8080
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# EC2 Instance
resource "aws_instance" "jenkins_server" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = var.instance_type
  key_name      = var.key_name
  vpc_security_group_ids = [aws_security_group.jenkins_sg.id]

  user_data = file("userdata.sh")

  tags = {
    Name = "Jenkins-Server"
  }
}

# Output Public IP
output "public_ip" {
  value = aws_instance.jenkins_server.public_ip
}

~
~
~
~
~
```

Initialize terraform

Terraform init

```
root@ip-172-31-26-239: /home/ubuntu/terraform-jenkins
root@ip-172-31-26-239:/home/ubuntu/terraform-jenkins# terraform init
Initializing provider plugins found in the configuration...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.49.0...
- Installed hashicorp/aws v6.49.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
root@ip-172-31-26-239:/home/ubuntu/terraform-jenkins#
```

→To validate the code we use below command

Terraform validate -- The terraform validate command is used to check whether the Terraform configuration files are syntactically valid and internally consistent.

```
root@ip-172-31-26-239:/home/ubuntu/terraform-jenkins# terraform validate
Success! The configuration is valid.
root@ip-172-31-26-239:/home/ubuntu/terraform-jenkins#
```

Preview the Infrastructure

Before creating any resources, run the following command to preview the infrastructure changes:

terraform plan -- The terraform plan command generates an execution plan and shows what Terraform will create, modify, or destroy based on the current configuration.

It shows

- EC2 Instance creation
- Security Group creation
- Any other AWS resources defined in the Terraform configuration

```
root@ip-172-31-26-239: /home/ubuntu/terraform-jenkins
```

```
root@ip-172-31-26-239:/home/ubuntu/terraform-jenkins# terraform plan
data.aws_ami.ubuntu: Reading...
data.aws_ami.ubuntu: Read complete after 0s [id=ami-0021ac0c2e69d9c55]
```

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

```
# aws_instance.jenkins_server will be created
+ resource "aws_instance" "jenkins_server" {
  + ami                    = "ami-0021ac0c2e69d9c55"
  + arn                   = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone      = (known after apply)
  + disable_api_stop      = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized          = (known after apply)
  + enable_primary_ipv6    = (known after apply)
  + force_destroy          = false
  + get_password_data      = false
  + host_id                = (known after apply)
  + host_resource_group_arn = (known after apply)
  + iam_instance_profile   = (known after apply)
  + id                     = (known after apply)
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_lifecycle     = (known after apply)
  + instance_state         = (known after apply)
  + instance_type          = "t2.small"
  + ipv6_address_count      = (known after apply)
  + ipv6_addresses         = (known after apply)
  + key_name               = "jen.pem"
  + monitoring              = (known after apply)
  + outpost_arn            = (known after apply)
  + password_data          = (known after apply)
  + placement_group        = (known after apply)
  + placement_group_id     = (known after apply)
  + placement_partition_number = (known after apply)
  + primary_network_interface_id = (known after apply)
  + private_dns            = (known after apply)
  + private_ip             = (known after apply)
  + public_dns             = (known after apply)
  + public_ip              = (known after apply)
  + region                 = "us-east-1"
  + secondary_private_ips   = (known after apply)
  + security_groups        = (known after apply)
  + source_dest_check       = true
```


root@ip-172-31-26-239: /home/ubuntu/terraform-jenkins

```
+ source_dest_check = true
+ spot_instance_request_id = (known after apply)
+ subnet_id = (known after apply)
+ tags = {
  + "Name" = "Jenkins-Server"
}
+ tags_all = {
  + "Name" = "Jenkins-Server"
}
+ tenancy = (known after apply)
+ user_data = <<-EOT
    #!/bin/bash
    # Update packages
    apt update -y
    # Install Java 21
    apt install openjdk-21-jdk -y
    # Create keyrings directory
    mkdir -p /etc/apt/keyrings
    # Add Jenkins key
    wget -O /etc/apt/keyrings/jenkins-keyring.asc \
    https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
    # Add Jenkins repository
    echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] \
    https://pkg.jenkins.io/debian-stable binary/" | sudo tee \
    /etc/apt/sources.list.d/jenkins.list > /dev/null
    # Update packages again
    apt update -y
    # Install Jenkins
    apt install jenkins -y
    # Enable and start Jenkins
    systemctl enable jenkins
    systemctl start jenkins
EOT
+ user_data_base64 = (known after apply)
+ user_data_replace_on_change = false
+ vpc_security_group_ids = (known after apply)

+ capacity_reservation_specification (known after apply)

+ cpu_options (known after apply)

+ ebs_block_device (known after apply)

+ enclave_options (known after apply)

+ ephemeral_block_device (known after apply)

+ instance_market_options (known after apply)
```

```
+ primary_network_interface (known after apply)
+ private_dns_name_options (known after apply)
+ root_block_device (known after apply)
+ secondary_network_interface (known after apply)
}

# aws_security_group.jenkins_sg will be created
+ resource "aws_security_group" "jenkins_sg" {
+   arn                = (known after apply)
+   description        = "Managed by Terraform"
+   egress              = [
+     {
+       + cidr_blocks    = [
+         + "0.0.0.0/0",
+       ]
+       + from_port      = 0
+       + ipv6_cidr_blocks = []
+       + prefix_list_ids = []
+       + protocol        = "-1"
+       + security_groups = []
+       + self             = false
+       + to_port         = 0
+       # (1 unchanged attribute hidden)
+     },
+   ]
+   id                  = (known after apply)
+   ingress              = [
+     {
+       + cidr_blocks    = [
+         + "0.0.0.0/0",
+       ]
+       + description    = "HTTP"
+       + from_port      = 80
+       + ipv6_cidr_blocks = []
+       + prefix_list_ids = []
+       + protocol        = "tcp"
+       + security_groups = []
+       + self             = false
+       + to_port         = 80
+     },
+   ]
+   {
+     + cidr_blocks    = [
+       + "0.0.0.0/0",
+     ]
+   }
+ }
```

root@ip-172-31-26-239: /home/ubuntu/terraform-jenkins

```
+ to_port      = 80
+ },
+ {
+   + cidr_blocks = [
+     + "0.0.0.0/0",
+   ]
+   + description = "Jenkins"
+   + from_port   = 8080
+   + ipv6_cidr_blocks = []
+   + prefix_list_ids = []
+   + protocol     = "tcp"
+   + security_groups = []
+   + self         = false
+   + to_port      = 8080
+ },
+ {
+   + cidr_blocks = [
+     + "0.0.0.0/0",
+   ]
+   + description = "SSH"
+   + from_port   = 22
+   + ipv6_cidr_blocks = []
+   + prefix_list_ids = []
+   + protocol     = "tcp"
+   + security_groups = []
+   + self         = false
+   + to_port      = 22
+ },
+ ],
+ name           = "jenkins-security-group"
+ name_prefix    = (known after apply)
+ owner_id       = (known after apply)
+ region         = "us-east-1"
+ revoke_rules_on_delete = false
+ tags_all       = (known after apply)
+ vpc_id         = (known after apply)
}
```

Plan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```
+ public_ip = (known after apply)
```

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
root@ip-172-31-26-239:/home/ubuntu/terraform-jenkins# |

Create the Infrastructure

After validating the configuration and reviewing the execution plan, create the infrastructure using the following command:

terraform apply

```
root@ip-172-31-26-239: /home/ubuntu/terraform-jenkins
```

```
root@ip-172-31-26-239:/home/ubuntu/terraform-jenkins# terraform apply
data.aws_ami.ubuntu: Reading...
aws_security_group.jenkins_sg: Refreshing state... [id=sg-045aaf716ce135233]
data.aws_ami.ubuntu: Read complete after 0s [id=ami-0021ac0c2e69d9c55]
```

Terraform used the selected providers to generate the following execution plan.

Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

```
# aws_instance.jenkins_server will be created
+ resource "aws_instance" "jenkins_server" {
+   ami                        = "ami-0021ac0c2e69d9c55"
+   arn                       = (known after apply)
+   associate_public_ip_address = (known after apply)
+   availability_zone         = (known after apply)
+   disable_api_stop         = (known after apply)
+   disable_api_termination   = (known after apply)
+   ebs_optimized             = (known after apply)
+   enable_primary_ipv6       = (known after apply)
+   force_destroy             = false
+   get_password_data         = false
+   host_id                   = (known after apply)
+   host_resource_group_arn    = (known after apply)
+   iam_instance_profile       = (known after apply)
+   id                        = (known after apply)
+   instance_initiated_shutdown_behavior = (known after apply)
+   instance_lifecycle         = (known after apply)
+   instance_state            = (known after apply)
+   instance_type             = "t3.micro"
+   ipv6_address_count         = (known after apply)
+   ipv6_addresses            = (known after apply)
+   key_name                   = "jen"
+   monitoring                 = (known after apply)
+   outpost_arn               = (known after apply)
+   password_data             = (known after apply)
+   placement_group           = (known after apply)
+   placement_group_id        = (known after apply)
+   placement_partition_number = (known after apply)
+   primary_network_interface_id = (known after apply)
+   private_dns               = (known after apply)
+   private_ip                 = (known after apply)
+   public_dns                 = (known after apply)
+   public_ip                  = (known after apply)
+   region                    = "us-east-1"
+   secondary_private_ips      = (known after apply)
+   security_groups            = (known after apply)
```

root@ip-172-31-26-239: /home/ubuntu/terraform-jenkins

```
+ secondary_private_ips      = (known after apply)
+ security_groups            = (known after apply)
+ source_dest_check          = true
+ spot_instance_request_id   = (known after apply)
+ subnet_id                  = (known after apply)
+ tags                        = {
  + "Name" = "Jenkins-Server"
}
+ tags_all                    = {
  + "Name" = "Jenkins-Server"
}
+ tenancy                     = (known after apply)
+ user_data                   = <<-EOT
  #!/bin/bash
  # Update packages
  apt update -y
  # Install Java 21
  apt install openjdk-21-jdk -y
  # Create keyrings directory
  mkdir -p /etc/apt/keyrings
  # Add Jenkins key
  wget -O /etc/apt/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
  # Add Jenkins repository
  echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/" | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null
  # Update packages again
  apt update -y
  # Install Jenkins
  apt install jenkins -y
  # Enable and start Jenkins
  systemctl enable jenkins
  systemctl start jenkins
EOT
+ user_data_base64            = (known after apply)
+ user_data_replace_on_change = false
+ vpc_security_group_ids      = [
  + "sg-045aaf716ce135233",
]

+ capacity_reservation_specification (known after apply)

+ cpu_options (known after apply)

+ ebs_block_device (known after apply)

+ enclave_options (known after apply)
```

root@ip-172-31-26-239: /home/ubuntu/terraform-jenkins

```
+ cpu_options (known after apply)
+ ebs_block_device (known after apply)
+ enclave_options (known after apply)
+ ephemeral_block_device (known after apply)
+ instance_market_options (known after apply)
+ maintenance_options (known after apply)
+ metadata_options (known after apply)
+ network_interface (known after apply)
+ primary_network_interface (known after apply)
+ private_dns_name_options (known after apply)
+ root_block_device (known after apply)
+ secondary_network_interface (known after apply)
}
```

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```
+ public_ip = (known after apply)
```

Do you want to perform these actions?

Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_instance.jenkins_server: Creating...

aws_instance.jenkins_server: Still creating... [00m10s elapsed]

aws_instance.jenkins_server: Creation complete after 13s [id=i-02780111aa991d96b]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

public_ip = "18.208.252.56"

root@ip-172-31-26-239:/home/ubuntu/terraform-jenkins# |

→To check the output

Terraform output

```
root@ip-172-31-26-239:/home/ubuntu/terraform-jenkins# terraform output
public_ip = "18.208.252.56"
root@ip-172-31-26-239:/home/ubuntu/terraform-jenkins# |
```

Instance is created

Instances (1/15) Info		Last updated less than a minute ago		Connect	Instance state ▼	Actions ▼	Launch instances ▼
Saved filter sets		Choose filter set ▼		Find Instance by attribute or tag (case-sensitive)		< 1 > ⚙	
	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	
☐	Jenkins-Server	i-02780111aa991d96b	Running	t3.micro	3/3 checks passed	us-east-1d	

Login using ssh

```
ubuntu@ip-172-31-32-141: ~
manis@MANISHAREDDY MINGW64 ~/Downloads
$ ssh -i "jen.pem" ubuntu@ec2-18-208-252-56.compute-1.amazonaws.com
The authenticity of host 'ec2-18-208-252-56.compute-1.amazonaws.com (18.208.252.56)' can't be established.
ED25519 key fingerprint is SHA256:OM6e/QBN2ybKtdWwaE2RUTCc8FPMNk3xdKk/qMNW9RY.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-18-208-252-56.compute-1.amazonaws.com' (ED25519)
to the list of known hosts.
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1017-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Sat Jun  6 11:22:02 UTC 2026

System load:  0.02           Temperature:   -273.1 C
Usage of /:   43.4% of 6.71GB Processes:      115
Memory usage: 56%           Users logged in: 0
Swap usage:   0%            IPv4 address for ens5: 172.31.32.141

Expanded Security Maintenance for Applications is not enabled.

1 update can be applied immediately.
1 of these updates is a standard security update.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

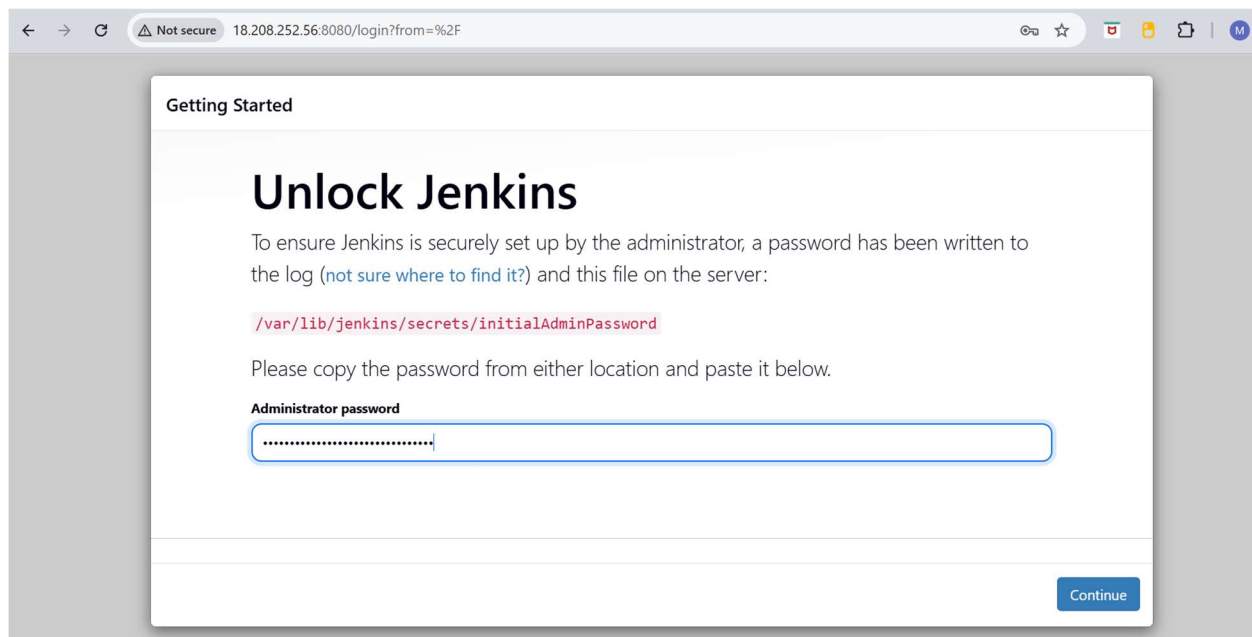
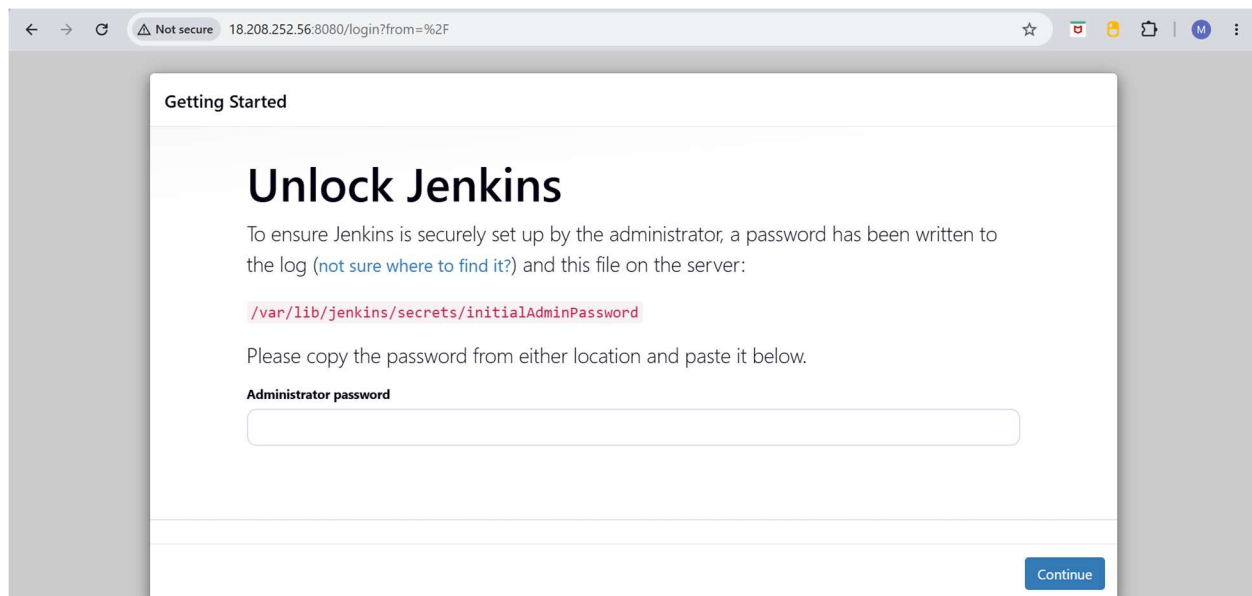
The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

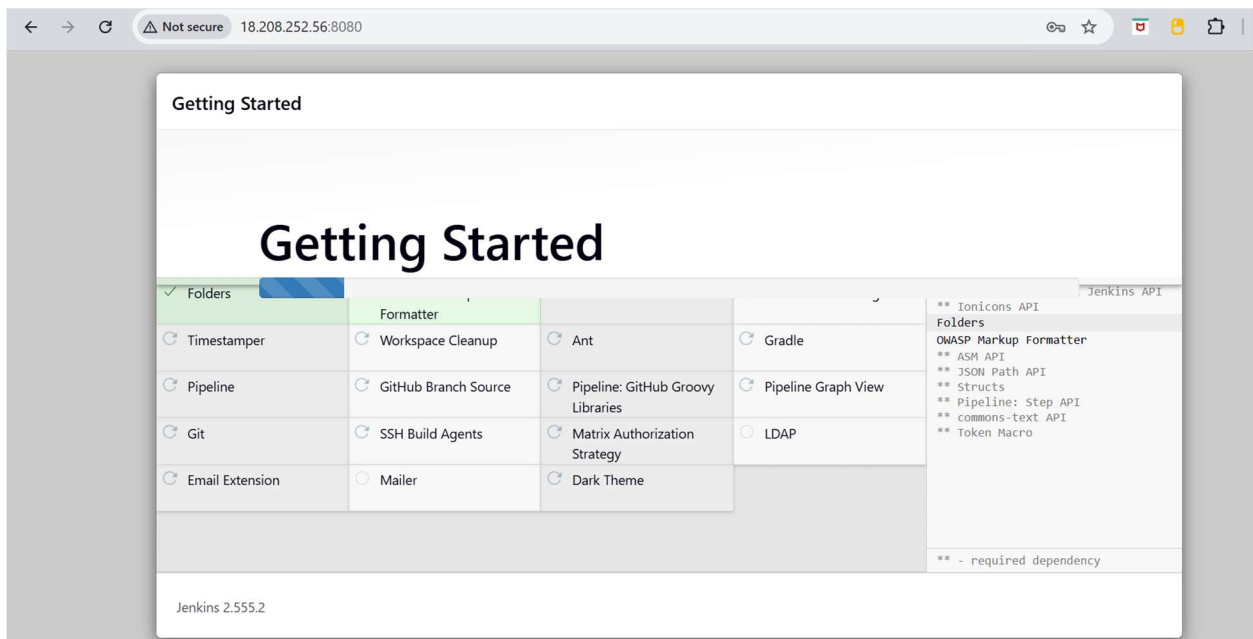
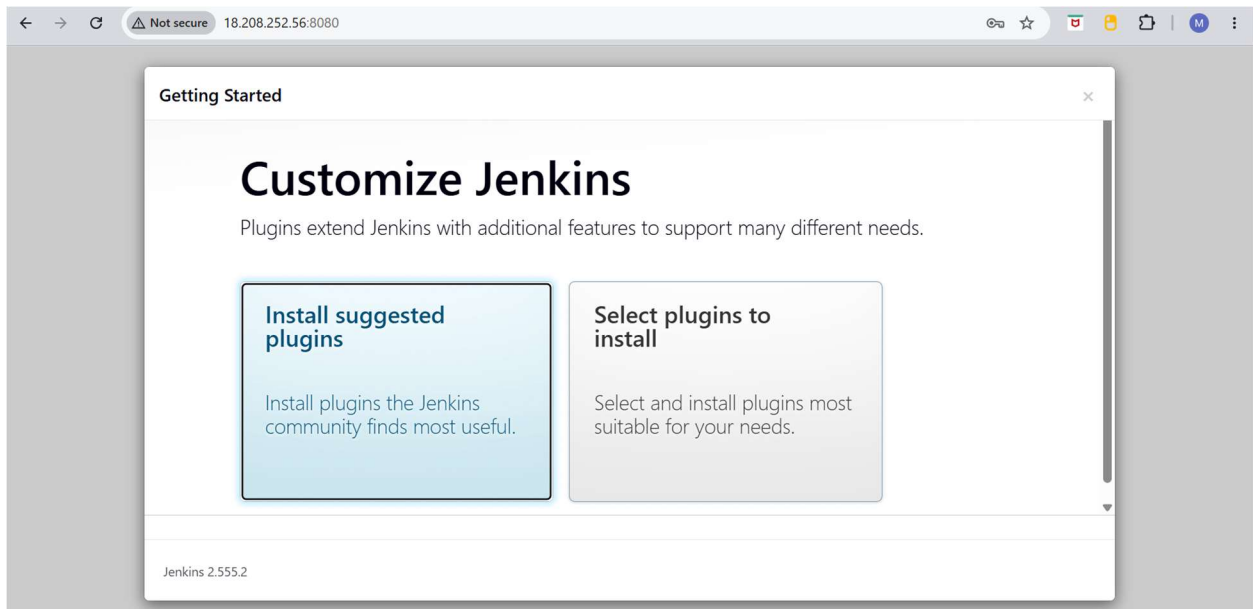
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

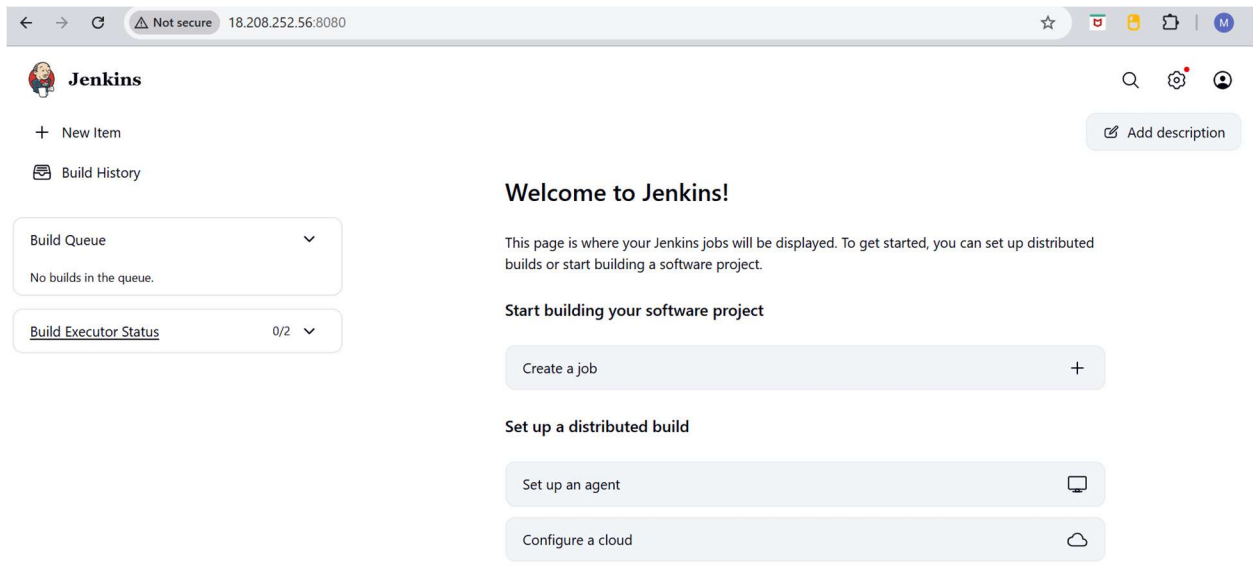
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-32-141:~$ |
```

Check with public ip:8080 whether the Jenkins is working or not







Errors which I have faced is

```
root@ip-172-31-26-239: /home/ubuntu/terraform-jenkins
+ secondary_network_interface (known after apply)
}
Plan: 1 to add, 0 to change, 0 to destroy.
Changes to Outputs:
+ public_ip = (known after apply)
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes
aws_instance.jenkins_server: Creating...
Error: creating EC2 Instance: operation error EC2: RunInstances, https response error StatusCode: 400, RequestID: d52ef63a-a55a-4a05-b735-f2e82e6241af, api error InvalidParameterCombination: The specified instance type is not eligible for Free Tier. For a list of Free Tier instance types, run 'describe-instance-types' with the filter 'free-tier-eligible=true'.

with aws_instance.jenkins_server,
  on main.tf line 54, in resource "aws_instance" "jenkins_server":
  54: resource "aws_instance" "jenkins_server" {
```

→Instance Type Not Eligible

I gave t2.micro which is being denied by aws. Later I have change to t3.micro it worked fine

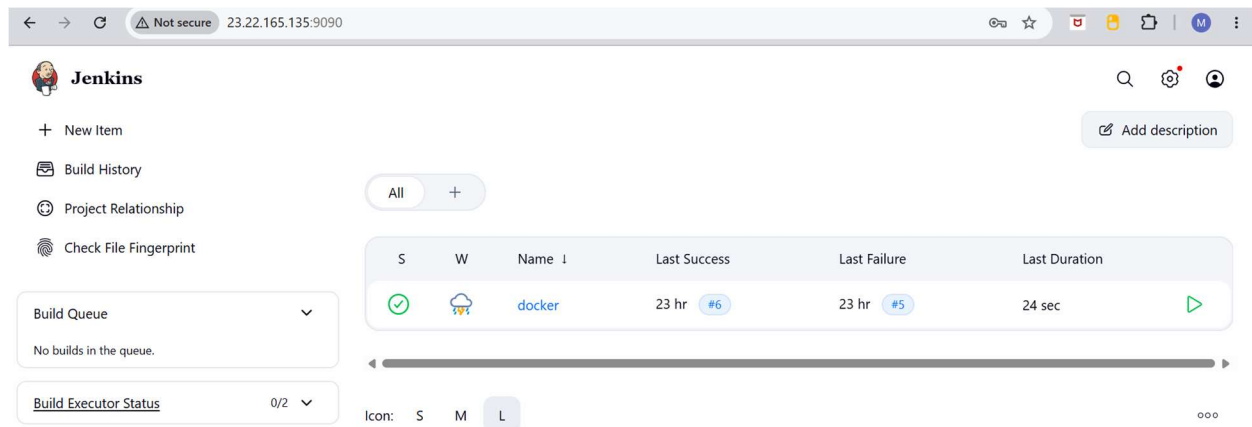
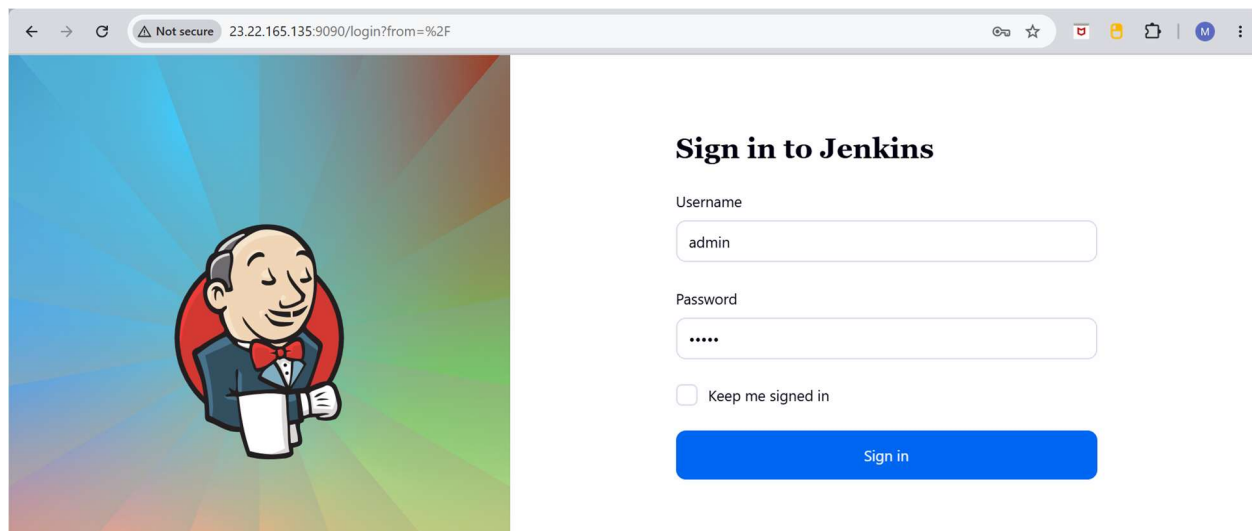
5.Create one Jenkins job to build and push the Docker image to Docker Hub.

Check whether you have jenkins server running

```
MINGW64:/c/Users/manis/Downloads
root@ip-172-31-26-239:/home/ubuntu# docker run -itd \
--name jenkins \
-p 8085:8080 \
-p 50000:50000 \
jenkins:2.60.3
Unable to find image 'jenkins:2.60.3' locally
2.60.3: Pulling from library/jenkins
9a8ea045c926: Pull complete
38430d93efed: Pull complete
1607093a898c: Pull complete
7e46ccda148a: Pull complete
d4eee24d4dac: Pull complete
c0cbcb5ac747: Pull complete
c58988e753d7: Pull complete
35ade7a86a8e: Pull complete
794a04897db9: Pull complete
aa433a6a56b1: Pull complete
70fcfa476f73: Pull complete
841c1dd38d62: Pull complete
0539c80a02be: Pull complete
b865dcb08714: Pull complete
54fefc6dcf80: Pull complete
5a3779030005: Pull complete
911bc90e47a8: Pull complete
12b47c68955c: Pull complete
1322ea3e7bfd: Pull complete
55cbf04beb70: Pull complete
Digest: sha256:eeb4850eb65f2d92500e421b430ed1ec58a7ac909e91f518926e02473904f668
Status: Downloaded newer image for jenkins:2.60.3
c1b1b1f29b09dd2d30048373dc8787cdf63a096a45f842e1a128e478a41879e1
```

```
root@ip-172-31-26-239:/home/ubuntu# docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED
STATUS        PORTS          NAMES
c1b1b1f29b09   jenkins:2.60.3                     "/bin/tini -- /usr/l..." About a minute ago
Created
11a2af8f506d   my-custom-nginx                     "/docker-entrypoint...." 22 hours ago
Exited (0) 19 hours ago
jenkins
b69d55e858b4   jenkins/jenkins:lts               "/usr/bin/tini -- /u..." 24 hours ago
Up 2 hours    0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp, 0.0.0.0:9090->8080/tcp, [::]:9090->8080/tcp jenkins-server
448c2a9bc57a   single-stage-app:v1               "/usr/local/bin/mvn-..." 25 hours ago
Created
single-stage-app
```

As we have jenkins, lets login into it



create credentials for Dockerhub

Generate access token in Dockerhub

DockerHub→Settings→personal access tokens→permissions(RWX) →copy 2→give in manage jenkins

manisha3299
Your personal account

Home

Hub

Build Cloud

Hardened Images

Scout

Testcontainers Cloud

Docker Desktop

Settings

Account information

Email

Password

2FA

Personal access tokens

Connected accounts

Personal access tokens / New access token

Create access token

A personal access token is similar to a password except you can have many tokens and revoke access to each one at any time. [Learn more](#)

Access token description

Expiration date

None

Optional

Access permissions

Public Repo Read-only

Public Repo Read-only tokens allow to view, search, and pull images from any public repositories.

Cancel

Generate

manisha3299
Your personal account

Home

Hub

Build Cloud

Hardened Images

Scout

Testcontainers Cloud

Docker Desktop

Settings

Account information

Email

Password

2FA

Personal access tokens / New access token

Create access token

A personal access token is similar to a password except you can have many tokens and revoke access to each one at any time. [Learn more](#)

Access token description

Jenkins

Expiration date

None

Optional

Access permissions

Read, Write, Delete

Read, Write, Delete tokens allow you to manage your repositories.

Cancel

Generate

The screenshot shows the Docker Hub account settings page for 'manisha3299'. The left sidebar contains navigation links: Home, Hub, Build Cloud, Hardened Images, Scout, Testcontainers Cloud, Docker Desktop, Settings, Account information, Email, Password, 2FA, Personal access tokens (selected), Connected accounts, Convert, and Privacy. The main content area is titled 'Copy access token' and includes instructions: 'Use this token as a password when you sign in from the Docker CLI client. [Learn more](#)'. It also states: 'Make sure you copy your personal access token now. Your personal access token is only displayed once. It isn't stored and can't be retrieved later.' Below this, the 'Access token description' is 'Jenkins', 'Expires on' is 'Never', and 'Access permissions' are 'Read, Write, Delete'. A section titled 'To use the access token from your Docker CLI client:' lists two steps: 1. Run the command `$ docker login -u manisha3299` (with a 'Copy' button), and 2. At the password prompt, enter the personal access token. The token is displayed as `dcKr_pat__DecJvkFNpYEjzX-Iq9W2bZS1Ig` (with a 'Copy' button). A green success message at the bottom says 'Successfully created personal access token Jenkins.' with a 'Back to access tokens' button.

Click on username and password

The screenshot shows the Jenkins 'Add Credentials' dialog box. The background shows the 'Credentials' page with a list of credentials, including 'dockerhub-creds' for 'manisha3299'. The dialog box has a title 'Add Credentials' and a close button. It prompts the user to 'Select a type of credential' and lists several options: 'Username with password' (selected), 'GitHub App', 'SSH Username with private key', 'Secret file', and 'Secret text'. The 'Username with password' option is highlighted with a blue border and includes a description: 'Commonly used for authentication to services like Git, APIs, or registries.' A 'Next' button is at the bottom of the dialog box.

The screenshot shows the Jenkins 'Credentials' page. The top navigation bar includes 'Jenkins', 'Manage Jenkins', and 'Credentials'. The main content area is titled 'Credentials' and features a '+ Add Credentials' button. Below this, a list of credentials is shown, including 'dockerhub-creds' for 'manisha3299/*****'. The credential is associated with the 'System' store and is a 'Global' credential. A 'dockerhub-creds' credential is also listed below it. The page footer shows 'REST API' and 'Jenkins 2.55.2'.

Create a pipeline

← → ↻ ⚠ Not secure 23.22.165.135:9090/view/all/newJob ☆ 📄 🗂 |

 **Jenkins** / All ▾ / New Item 🔍 ⚙️

New Item

Enter an item name

Docker-Jenkins

Select an item type

 **Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

 **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

 **Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

OK

Write a pipeline script

```
pipeline {  
  agent any  
  
  environment {  
    IMAGE_NAME = "manisha3299/python-app:v1"  
  }  
  
  stages {  
  
    stage('Clone Code') {  
      steps {  
        git branch: 'main',  
          url: 'https://github.com/manisha-a99/Python-app.git'  
      }  
    }  
  }  
}
```

```
stage('Build Docker Image') {  
    steps {  
        sh 'docker build -t $IMAGE_NAME .'    }  
}
```

```
stage('Docker Login') {  
    steps {  
        withCredentials([  
            usernamePassword(  
                credentialsId: 'dockerhub-creds',  
                usernameVariable: 'DOCKER_USER',  
                passwordVariable: 'DOCKER_PASS'  
            )  
        ]){  
            sh '''  
            echo $DOCKER_PASS | docker login \  
            -u $DOCKER_USER \  
            --password-stdin  
            '''  
        }  
    }  
}
```

```
stage('Push Image') {
```

```
    steps {  
        sh 'docker push $IMAGE_NAME'  
    }  
}  
}  
  
post {  
    success {  
        echo 'Docker image pushed successfully!'  
    }  
  
    failure {  
        echo 'Pipeline failed!'  
    }  
}  
}
```

Pipeline script

Script ?

```
1 pipeline {  
2     agent any  
3  
4     environment {  
5         IMAGE_NAME = "manisha3299/python-app:v1"  
6     }  
7  
8     stages {  
9  
10        stage('Clone Code') {  
11            steps {  
12                git branch: 'main',  
13                url: 'https://github.com/manisha-a99/Python-app.git'  
14            }  
15        }  
16    }  
17 }
```

Pipeline script

Script ?

```
17 stage('Build Docker Image') {
18     steps {
19         sh 'docker build -t $IMAGE_NAME .'
20     }
21 }
22
23 stage('Docker Login') {
24     steps {
25         withCredentials([
26             usernamePassword(
27                 credentialsId: 'dockerhub-creds',
28                 usernameVariable: 'DOCKER_USER',
29                 passwordVariable: 'DOCKER_PASS'
30             )
31         ]) {
```

Pipeline script

Script ?

```
32         sh '''
33             echo $DOCKER_PASS | docker login \
34             -u $DOCKER_USER \
35             --password-stdin
36             '''
37         }
38     }
39 }
40
41 stage('Push Image') {
42     steps {
43         sh 'docker push $IMAGE_NAME'
44     }
45 }
46 }
```


Pipeline script

Script ?

```
43         sh 'docker push $IMAGE_NAME'
44     }
45 }
46 }
47
48 post {
49     success {
50         echo 'Docker image pushed successfully!'
51     }
52
53     failure {
54         echo 'Pipeline failed!'
55     }
56 }
57 }
```

Save

Apply

→ Job ran successfully

← → ↻ ⚠ Not secure 23.22.165.135:9090/job/Docker-Jenkins/3/ ☆ 📄 📁 📂 📌 M ⋮

 **Jenkins** / Docker-Jenkins ▾ / #3 🔍 ⚙️ 👤

 Status

 Changes

 Console Output

 Edit Build Information

 Delete build '#3'

 Timings

 Git Build Data

 Pipeline Overview

 Restart from Stage

 Replay

✅ #3 (06-Jun-2026, 1:41:02 pm) [Add description](#) [Keep this build forever](#)

 Started by user [admin](#) Started 15 sec ago
Took **9.6 sec**

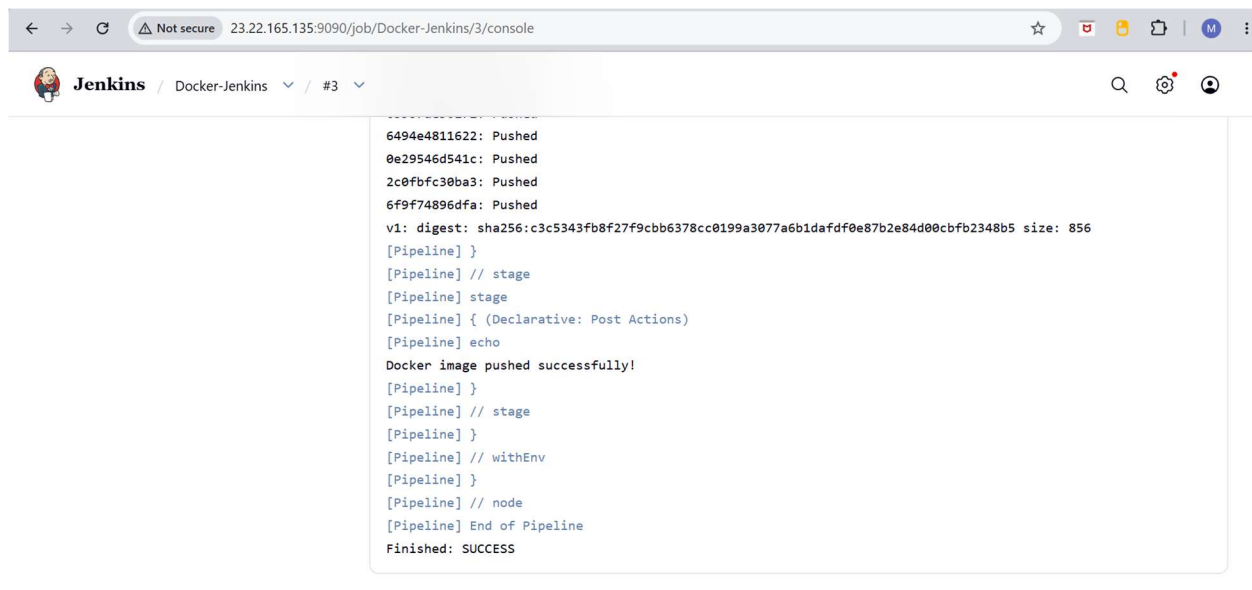
 This run spent:

- 11 ms waiting;
- 9.6 sec build duration;
- 9.6 sec total from scheduled to completion.

 **git** **Revision:** a9c18a887f134751f06bab9eb128a4387b42c141
Repository: <https://github.com/manisha-a99/Python-app.git>

- refs/remotes/origin/main

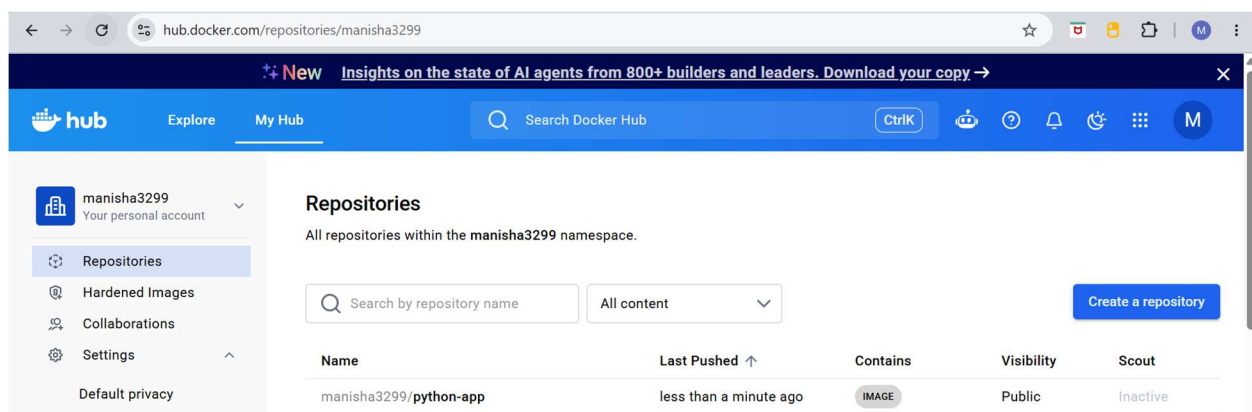
 No changes.



The screenshot shows the Jenkins web interface with a console log for a build named 'Docker-Jenkins'. The log displays the output of a Docker build and push command. It shows the image being pushed is '6494e4811622' and the digest is 'sha256:c3c5343fb8f27f9cbb6378cc0199a3077a6b1dafdf0e87b2e84d00cbfb2348b5'. The build is successful, and the image is pushed to Docker Hub.

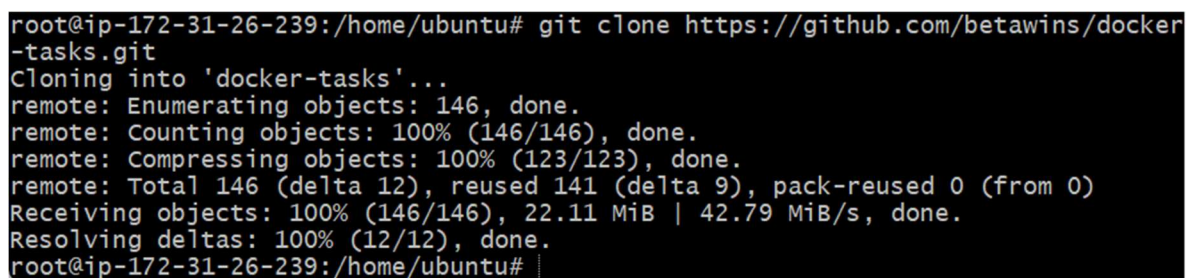
```
6494e4811622: Pushed
0e29546d541c: Pushed
2c0fbfc30ba3: Pushed
6f9f74896dfa: Pushed
v1: digest: sha256:c3c5343fb8f27f9cbb6378cc0199a3077a6b1dafdf0e87b2e84d00cbfb2348b5 size: 856
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Docker image pushed successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Successfully pushed image to docker hub



→From the frontend source code, write a Dockerfile, build a Docker image, run it, and push that image to your Docker registry.

First lets clone the code to our local machine



```
root@ip-172-31-26-239:/home/ubuntu# git clone https://github.com/betawins/docker-tasks.git
Cloning into 'docker-tasks'...
remote: Enumerating objects: 146, done.
remote: Counting objects: 100% (146/146), done.
remote: Compressing objects: 100% (123/123), done.
remote: Total 146 (delta 12), reused 141 (delta 9), pack-reused 0 (from 0)
Receiving objects: 100% (146/146), 22.11 MiB | 42.79 MiB/s, done.
Resolving deltas: 100% (12/12), done.
root@ip-172-31-26-239:/home/ubuntu#
```

ls→cd docker-tasks→ls→cd frontend_based_source→ls

```
root@ip-172-31-26-239:/home/ubuntu# ls
Dockerfile  docker-tasks  multi-stage-example  terraform-jenkins
compose-demo  index.html  nginx-custom-v2.tar  tomcat-alpine
root@ip-172-31-26-239:/home/ubuntu# cd docker-tasks/
root@ip-172-31-26-239:/home/ubuntu/docker-tasks# ls
Frontend_based_source  Java_based_source  NodeJs_based_source  README.md
root@ip-172-31-26-239:/home/ubuntu/docker-tasks# cd Frontend_based_source/
root@ip-172-31-26-239:/home/ubuntu/docker-tasks/Frontend_based_source# ls
index.html  javascript.js  style.css  todayDeal.js
root@ip-172-31-26-239:/home/ubuntu/docker-tasks/Frontend_based_source# |
```

Create dockerfile

This docker file downloads the latest nginx image and copies all the files from current directory to /usr/share/nginx/html

```
root@ip-172-31-26-239:/home/ubuntu/docker-tasks/Frontend_
FROM nginx:alpine
COPY . /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
~
~
~
```

Now lets build the image

docker build -t frontend-app:v1 .

```

root@ip-172-31-26-239: /home/ubuntu/docker-tasks/Frontend_based_source
root@ip-172-31-26-239: /home/ubuntu/docker-tasks/Frontend_based_source# docker build -t frontend-app:v1 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
                Install the buildx component to build images with BuildKit:
                https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 44.03kB
Step 1/4 : FROM nginx:alpine
alpine: Pulling from library/nginx
fbaed3f7fcbe: Pulling fs layer
abaae85d1626: Pulling fs layer
43f834d60d8a: Pulling fs layer
a5008f4a4b25: Pulling fs layer
de1b677d8c00: Pulling fs layer
94d083cf706a: Pulling fs layer
e654dbbbb9e1: Pulling fs layer
a5008f4a4b25: Download complete
ab556c9e4280: Download complete
2d8223aa4623: Download complete
e654dbbbb9e1: Download complete
de1b677d8c00: Download complete
abaae85d1626: Download complete
43f834d60d8a: Download complete
94d083cf706a: Download complete
de1b677d8c00: Pull complete
abaae85d1626: Pull complete
43f834d60d8a: Pull complete
fbaed3f7fcbe: Download complete
a5008f4a4b25: Pull complete
e654dbbbb9e1: Pull complete
94d083cf706a: Pull complete
fbaed3f7fcbe: Pull complete
Digest: sha256:8b1e78743a03dbb2c95171cc58639fef29abc8816598e27fb910ed2e621e589a
Status: Downloaded newer image for nginx:alpine
----> 8b1e78743a03
Step 2/4 : COPY . /usr/share/nginx/html
----> a5d3b65ee03c
Step 3/4 : EXPOSE 80
----> Running in 3989f8512fe4
----> Removed intermediate container 3989f8512fe4
----> c1ea9126b80e
Step 4/4 : CMD ["nginx", "-g", "daemon off;"]
----> Running in 1eab6f913544
----> Removed intermediate container 1eab6f913544
----> 7f4cdb4d810d
Successfully built 7f4cdb4d810d
Successfully tagged frontend-app:v1

```

View docker images

docker images

```

root@ip-172-31-26-239: /home/ubuntu/docker-tasks/Frontend_based_source# docker images

```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
142083399660.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx:latest	96fa51fbcc00	238MB	63.1MB	U
142083399660.dkr.ecr.us-east-1.amazonaws.com/nginx-custom:v1	2ad06826471f	584MB	193MB	
142083399660.dkr.ecr.us-east-1.amazonaws.com/nginx-scan-test:latest	d679e3e7dc30	238MB	63.1MB	
alpine:latest	5b10f432ef3d	13.1MB	3.95MB	
amazoncorretto:17	823513044539	711MB	221MB	
amazonlinux:latest	ad882807c6c8	227MB	59.3MB	
frontend-app:v1	7f4cdb4d810d	92.8MB	26.1MB	
jenkins/jenkins:lts	01e392ffef89	799MB	269MB	U
jenkins:2.60.3	eeb4850eb65f	1.08GB	316MB	U
manish3298/my-custom-nginx:latest	96fa51fbcc00	238MB	63.1MB	U

Now run the container

docker run -d --name frontend-container -p 8082:80 frontend-app:v1

```

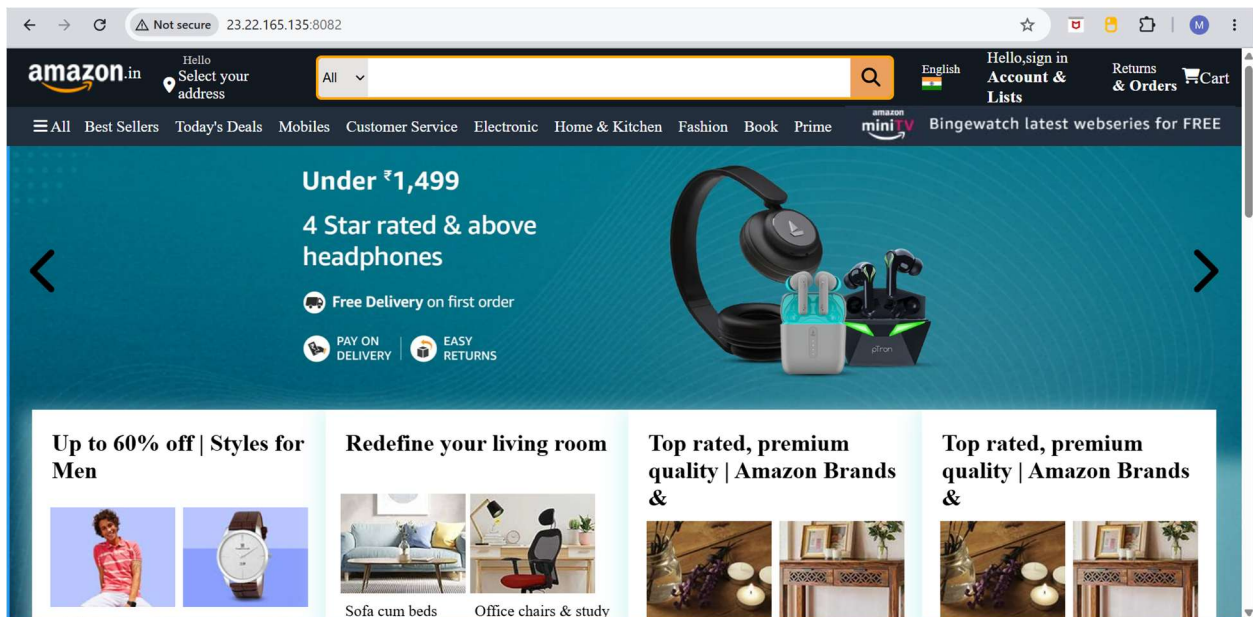
root@ip-172-31-26-239: /home/ubuntu/docker-tasks/Frontend_based_source# docker run -d --name frontend-container -p 8082:80 frontend-app:v1
d025661f78411ee5d5f2188c9d308f5e2e508307f4b470c1d8e9af713cd98140
root@ip-172-31-26-239: /home/ubuntu/docker-tasks/Frontend_based_source#

```

Docker ps

```
root@ip-172-31-26-239:/home/ubuntu/docker-tasks/Frontend_based_source# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
MES
d025661f7841   frontend-app:v1                     "/docker-entrypoint..." About a minute ago Up About a minute 0.0.0.0:8082->80/tcp, [::]:8082->80/tcp
frontend-container
b69d55e858b4   jenkins/jenkins:lts                "/usr/bin/tini -- /u..." 26 hours ago   Up 5 hours    0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp, 0.0.0.0:9090->8080/tcp, [::]:9090->8080/tcp
jenkins-server
root@ip-172-31-26-239:/home/ubuntu/docker-tasks/Frontend_based_source#
```

Lets verify in the browser



Now we need to push to docker hub

→First lets create a repo

hub.docker.com/repository/create?namespace=manisha3299

Your personal account

- Repositories
- Hardened Images
- Collaborations
- Settings
- Default privacy
- Notifications
- Billing
- Usage
- Pulls
- Storage

Create repository

Repository Name *
frontend-app

Short description

A short description to identify your repository. If the repository is public, this description is used to index your content on Docker Hub and in search engines, and is visible to users in search results.

Visibility

Using 0 of 1 private repositories. [Upgrade your plan to add more](#)

☒ Public
Appears in Docker Hub search results

☐ Private
Only visible to you

[Cancel](#) [Create](#)

Pushing images

You can push a new image to this repository using the CLI:

```
docker tag local-image:tagname new-repo:tagname
docker push new-repo:tagname
```

Make sure to replace `tagname` with your desired image repository tag.

[Upgrade plan](#)

docker tag frontend-app:v1 manisha3299/frontend-app:v1

docker push manisha3299/frontend-app:v1

```
root@ip-172-31-26-239:/home/ubuntu/docker-tasks/Frontend_based_source# docker tag frontend-app:v1 manisha3299/frontend-app:v1
root@ip-172-31-26-239:/home/ubuntu/docker-tasks/Frontend_based_source# docker push manisha3299/frontend-app:v1
The push refers to repository [docker.io/manisha3299/frontend-app]
43f834d60d8a: Mounted from library/nginx
de1b677d8c00: Mounted from library/nginx
e654dbbbb9e1: Mounted from library/nginx
abaae85d1626: Mounted from library/nginx
3fbdade2f52f: Pushed
94d083cf706a: Mounted from library/nginx
a5008f4a4b25: Mounted from library/nginx
6a0ac1617861: Mounted from library/alpine
fbaed3f7fcbe: Mounted from library/nginx
v1: digest: sha256:7f4cdb4d810d8b276cfc26a54fb16f7515dd23eb55d8bcf53651bf7486e623b0 size: 2262
root@ip-172-31-26-239:/home/ubuntu/docker-tasks/Frontend_based_source#
```

→ Verified in docker hub and image is pushed successfully

hub.docker.com/repositories/manisha3299

New Insights on the state of AI agents from 800+ builders and leaders. Download your copy →

hub Explore My Hub Search Docker Hub CtrlK

manisha3299 Your personal account

- Repositories
- Hardened Images
- Collaborations
- Settings
- Default privacy

Repositories

All repositories within the manisha3299 namespace.

Search by repository name All content Create a repository

Name	Last Pushed ↑	Contains	Visibility	Scout
manisha3299/frontend-app	1 minute ago	IMAGE	Public	Inactive

→ From the Java-based source code, write a Dockerfile, build, run, and push the image to the Docker registry.

→ let's do it for the Java-based code

→ Go to Java-based code directory

```
root@ip-172-31-26-239:/home/ubuntu/docker-tasks/Frontend_based_source# cd ..
root@ip-172-31-26-239:/home/ubuntu/docker-tasks# ls
Frontend_based_source  Java_based_source  NodeJs_based_source  README.md
root@ip-172-31-26-239:/home/ubuntu/docker-tasks# cd Java_based_source/
root@ip-172-31-26-239:/home/ubuntu/docker-tasks/Java_based_source# ls
pom.xml  src
root@ip-172-31-26-239:/home/ubuntu/docker-tasks/Java_based_source#
```

Create a Dockerfile

```
root@ip-172-31-26-239:/home/ubuntu/docker-tasks/Java_based_source
```

```
FROM maven:3.9.6-eclipse-temurin-17 AS build

WORKDIR /app

COPY . .

RUN mvn clean package -DskipTests

# Runtime image
FROM eclipse-temurin:17-jdk-alpine

WORKDIR /app

COPY --from=build /app/target/*.jar app.jar

EXPOSE 8080

CMD ["java", "-jar", "app.jar"]

~
~
~
```

Now let's build the image

docker build -t java-app:v1 .

root@ip-172-31-26-239: /home/ubuntu/docker-tasks/Java_based_source

root@ip-172-31-26-239:/home/ubuntu/docker-tasks/Java_based_source# docker build -t java-app:v1 .

DEPRECATED: The legacy builder is deprecated and will be removed in a future release.

Install the buildx component to build images with BuildKit:

<https://docs.docker.com/go/buildx/>

Sending build context to Docker daemon 1.925MB

Step 1/9 : FROM maven:3.9.6-eclipse-temurin-17 AS build

3.9.6-eclipse-temurin-17: Pulling from library/maven

5e5d1bccc544: Pulling fs layer

a3ba11f7aaae: Pulling fs layer

d59fd278c1b4: Pulling fs layer

67f99c2668af: Pulling fs layer

4a023cab5400: Pulling fs layer

fc06d68d71ba: Pulling fs layer

45f480637770: Pulling fs layer

58c3491a14eb: Pulling fs layer

4712dfa85971: Pulling fs layer

c97285723537: Pulling fs layer

c97285723537: Download complete

fc06d68d71ba: Download complete

58c3491a14eb: Download complete

a3ba11f7aaae: Download complete

4712dfa85971: Download complete

45f480637770: Download complete

5e5d1bccc544: Download complete

67f99c2668af: Download complete

4a023cab5400: Download complete

4a023cab5400: Pull complete

5e5d1bccc544: Pull complete

d59fd278c1b4: Download complete

c97285723537: Pull complete

a3ba11f7aaae: Pull complete

d59fd278c1b4: Pull complete

67f99c2668af: Pull complete

58c3491a14eb: Pull complete

45f480637770: Pull complete

fc06d68d71ba: Pull complete

4712dfa85971: Pull complete

Digest: sha256:29a1658b1f3078e07c2b17f7b519b45eb47f65d9628e887eac45a8c5c8f939d4

Status: Downloaded newer image for maven:3.9.6-eclipse-temurin-17

---> 29a1658b1f30

Step 2/9 : WORKDIR /app

---> Running in 4b228ed08be2

---> Removed intermediate container 4b228ed08be2

---> 99816c7a2ebf

Step 3/9 : COPY . .

---> c5e637540228

Step 4/9 : RUN mvn clean package -DskipTests

```

root@ip-172-31-26-239: /home/ubuntu/docker-tasks/Java_based_source
305 kB at 1.8 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/com/google/guava/guava/19.0/guava-19.0
.jar (2.3 MB at 12 MB/s)
[INFO] Replacing main artifact with repackaged archive
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 13.239 s
[INFO] Finished at: 2026-06-06T14:25:20Z
[INFO] -----
--> Removed intermediate container 615309025fd7
--> bf98da43b574
Step 5/9 : FROM eclipse-temurin:17-jdk-alpine
17-jdk-alpine: Pulling from library/eclipse-temurin
665e255b26ac: Pulling fs layer
7b50ae58d435: Pulling fs layer
ede0d10cee20: Pulling fs layer
b97e55ba8d37: Pulling fs layer
b97e55ba8d37: Download complete
0ff8dc90c5e8: Download complete
665e255b26ac: Download complete
ae12e7871527: Download complete
7b50ae58d435: Download complete
7b50ae58d435: Pull complete
ede0d10cee20: Download complete
665e255b26ac: Pull complete
ede0d10cee20: Pull complete
b97e55ba8d37: Pull complete
Digest: sha256:5d14725f0e49e19df217f6ce179039f01ca25f5f9aa958573b467312599ca246
Status: Downloaded newer image for eclipse-temurin:17-jdk-alpine
--> 5d14725f0e49
Step 6/9 : WORKDIR /app
--> Running in 9714b3ab0c55
--> Removed intermediate container 9714b3ab0c55
--> 684590965097
Step 7/9 : COPY --from=build /app/target/*.jar app.jar
--> db605c436930
Step 8/9 : EXPOSE 8080
--> Running in cee77dcc5873
--> Removed intermediate container cee77dcc5873
--> d8241eb60415
Step 9/9 : CMD ["java", "-jar", "app.jar"]
--> Running in a9f623aafa37
--> Removed intermediate container a9f623aafa37
--> 77d00b0ae7a7
Successfully built 77d00b0ae7a7
Successfully tagged java-app:v1
root@ip-172-31-26-239: /home/ubuntu/docker-tasks/Java_based_source#

```

→ Run the container

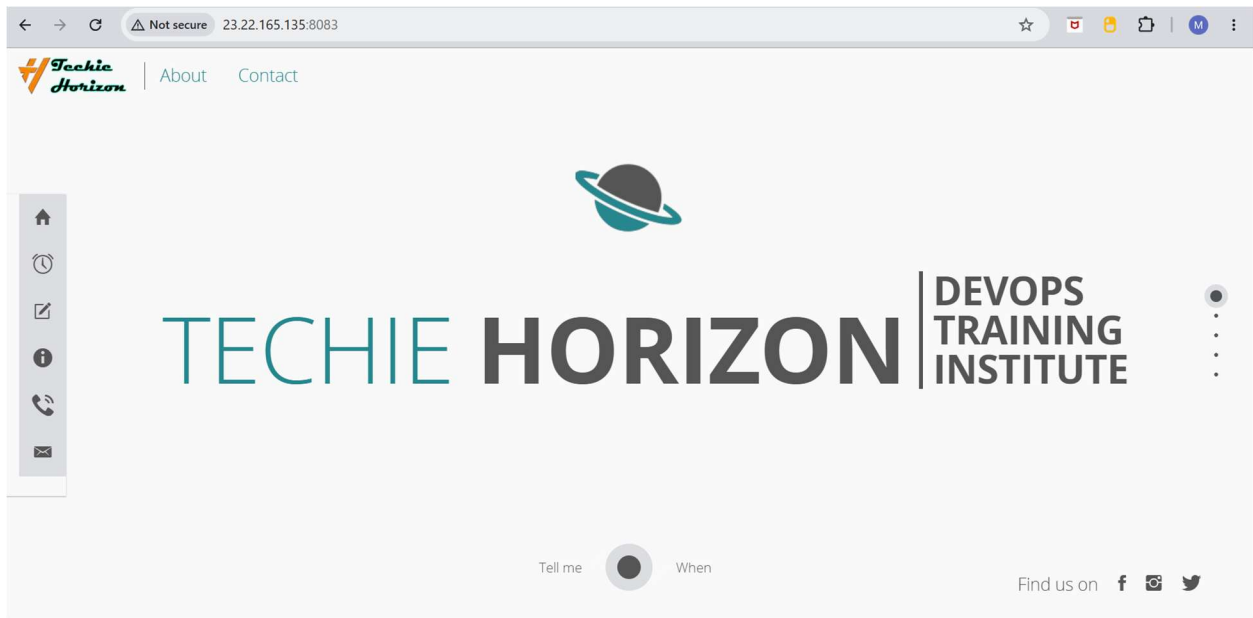
docker run -d --name java-container -p 8083:8080 java-app:v1

```

root@ip-172-31-26-239: /home/ubuntu/docker-tasks/Java_based_source# docker run -d --name java-contain
2e54f53bdf7be82f263f75132cbd034fe7dc51291dc593e1f9f810dd697fb585
root@ip-172-31-26-239: /home/ubuntu/docker-tasks/Java_based_source#

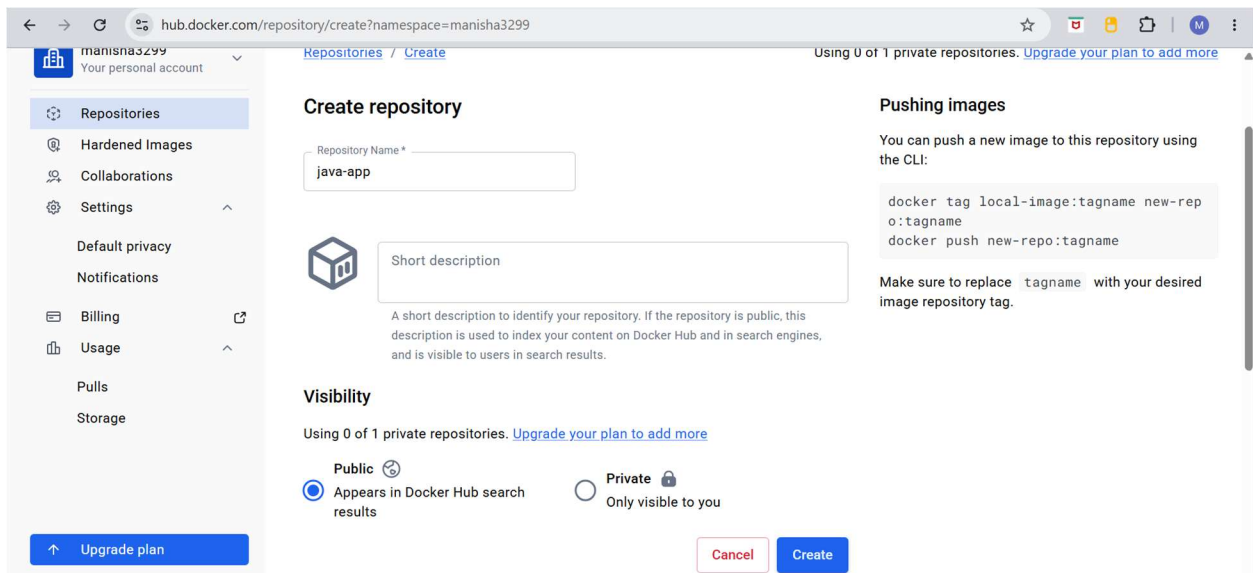
```

→ we are able to access the browser successfully



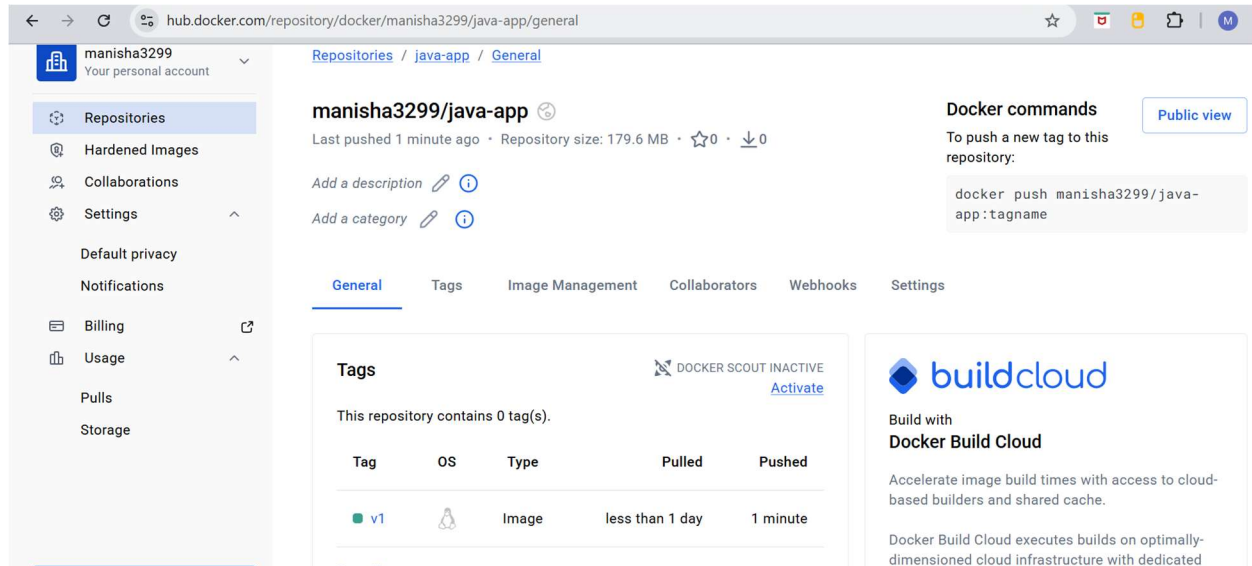
→ Now push to docker hub

First lets create a repo



```
root@ip-172-31-26-239:/home/ubuntu/docker-tasks/Java_based_source# docker tag java-app:v1 manisha3299/java-app:v1
root@ip-172-31-26-239:/home/ubuntu/docker-tasks/Java_based_source# docker push manisha3299/java-app:v1
The push refers to repository [docker.io/manisha3299/java-app]
6a0ac1617861: Mounted from manisha3299/frontend-app
7b50ae58d435: Mounted from library/eclipse-temurin
ede0d10cee20: Mounted from library/eclipse-temurin
c6b8fecff7ad: Pushed
665e255b26ac: Mounted from library/eclipse-temurin
92e4b1ac34ee: Pushed
b97e55ba8d37: Mounted from library/eclipse-temurin
v1: digest: sha256:77d00b0ae7a7ea97d771c997169d1b9cb61ab1e615824a2307bc40e2d6759246 size: 1837
root@ip-172-31-26-239:/home/ubuntu/docker-tasks/Java_based_source#
```

Image pushed successfully



manisha3299 / java-app / General

manisha3299/java-app

Last pushed 1 minute ago · Repository size: 179.6 MB · ☆0 · ↓0

Add a description ⓘ

Add a category ⓘ

Docker commands

To push a new tag to this repository:

```
docker push manisha3299/java-app:tagname
```

Public view

General Tags Image Management Collaborators Webhooks Settings

Tags

This repository contains 0 tag(s).

Tag	OS	Type	Pulled	Pushed
v1	linux	Image	less than 1 day	1 minute

DOCKER SCOUT INACTIVE [Activate](#)

buildcloud

Build with Docker Build Cloud

Accelerate image build times with access to cloud-based builders and shared cache.

Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated

→ From the Node.js-based source code, write a Dockerfile, build with tag v1, run, and push it to the Docker registry.

-First let's get into node.js

```
root@ip-172-31-26-239:/home/ubuntu/docker-tasks/NodeJs_based_source# cd ..
root@ip-172-31-26-239:/home/ubuntu/docker-tasks# ls
Frontend_based_source  Java_based_source  NodeJs_based_source  README.md
root@ip-172-31-26-239:/home/ubuntu/docker-tasks# cd NodeJs_based_source/
root@ip-172-31-26-239:/home/ubuntu/docker-tasks/NodeJs_based_source# ls
package-lock.json  package.json  public  src
root@ip-172-31-26-239:/home/ubuntu/docker-tasks/NodeJs_based_source#
```

→ Create a dockerfile

FROM node:18-alpine

WORKDIR /app

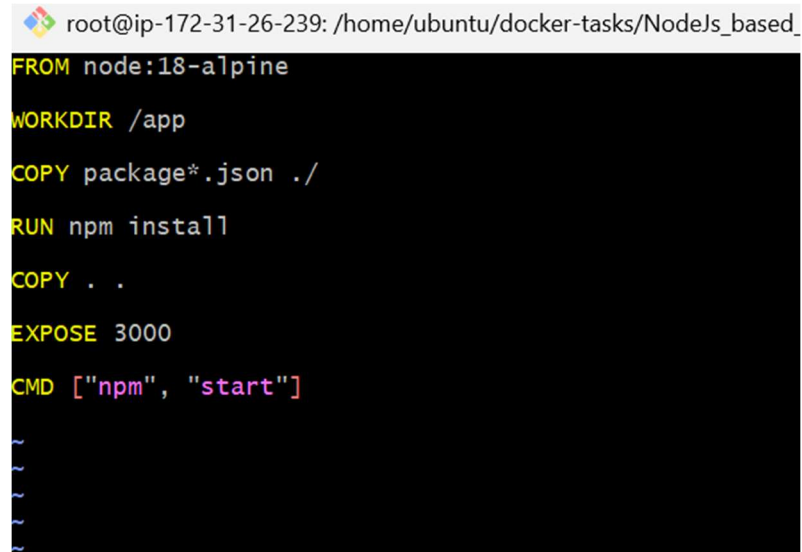
COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["npm", "start"]

A terminal window with a dark background and light-colored text. The prompt is 'root@ip-172-31-26-239: /home/ubuntu/docker-tasks/NodeJs_based_'. The Dockerfile content is as follows:

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

Below the Dockerfile content, there are four tilde (~) characters on separate lines.

This Dockerfile is used to containerize a Node.js application using a lightweight Alpine Linux-based Node.js image. It starts by pulling the node:18-alpine image, which includes Node.js version 18 and npm. The WORKDIR /app instruction creates and sets /app as the working directory inside the container. The COPY package*.json ./ command copies the package.json and package-lock.json files into the container, allowing dependencies to be installed before copying the entire source code. The RUN npm install command installs all the application dependencies defined in the package files. Next, COPY . . copies the remaining application source code into the container. The EXPOSE 3000 instruction indicates that the application listens on port 3000. Finally, the CMD ["npm", "start"] command starts the Node.js application when the container launches by executing the start script defined in the package.json file.

→ Now let's build an image

docker build -t node-app:v1 .


```

root@ip-172-31-26-239:/home/ubuntu/docker-tasks/NodeJs_based_source# docker build -t node-app:v1
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon   4.17MB
Step 1/7 : FROM node:18-alpine
18-alpine: Pulling from library/node
25ff2da83641: Pulling fs layer
dd71dde834b5: Pulling fs layer
1e5a4c89cee5: Pulling fs layer
f18232174bc9: Pulling fs layer
25ff2da83641: Download complete
1e5a4c89cee5: Download complete
f18232174bc9: Download complete
0758d11b0bb7: Download complete
f18232174bc9: Pull complete
27e1cfd0b5f2: Download complete
dd71dde834b5: Download complete
dd71dde834b5: Pull complete
25ff2da83641: Pull complete
1e5a4c89cee5: Pull complete
Digest: sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
Status: Downloaded newer image for node:18-alpine
--> 8d6421d663b4
Step 2/7 : WORKDIR /app
--> Running in c0930d13ad9d
--> Removed intermediate container c0930d13ad9d
--> c8663f08d53c
Step 3/7 : COPY package*.json /
--> c8663f08d53c
Step 4/7 : RUN npm install
--> Running in 742098d039aa
--> Removed intermediate container 742098d039aa
--> 161f12a1547e
Step 5/7 : COPY . .
--> 61abd6ea918e
Step 6/7 : EXPOSE 3000
--> Running in 742098d039aa
--> Removed intermediate container 742098d039aa
--> 161f12a1547e
Step 7/7 : CMD ["npm", "start"]
--> Running in 0ee72b04dd33
--> Removed intermediate container 0ee72b04dd33
--> 68ab2a1ea75c
Successfully built 68ab2a1ea75c
Successfully tagged node-app:v1
root@ip-172-31-26-239:/home/ubuntu/docker-tasks/NodeJs_based_source#

```

Docker images

nginx:alpine	8b1e78743a03	93.6MB	27MB
nginx:latest	5aca99593157	241MB	66MB
node-app:v1	68ab2a1ea75c	993MB	169MB
node:18-alpine	8d6421d663b4	181MB	45.3MB

Run docker container on 3000 port

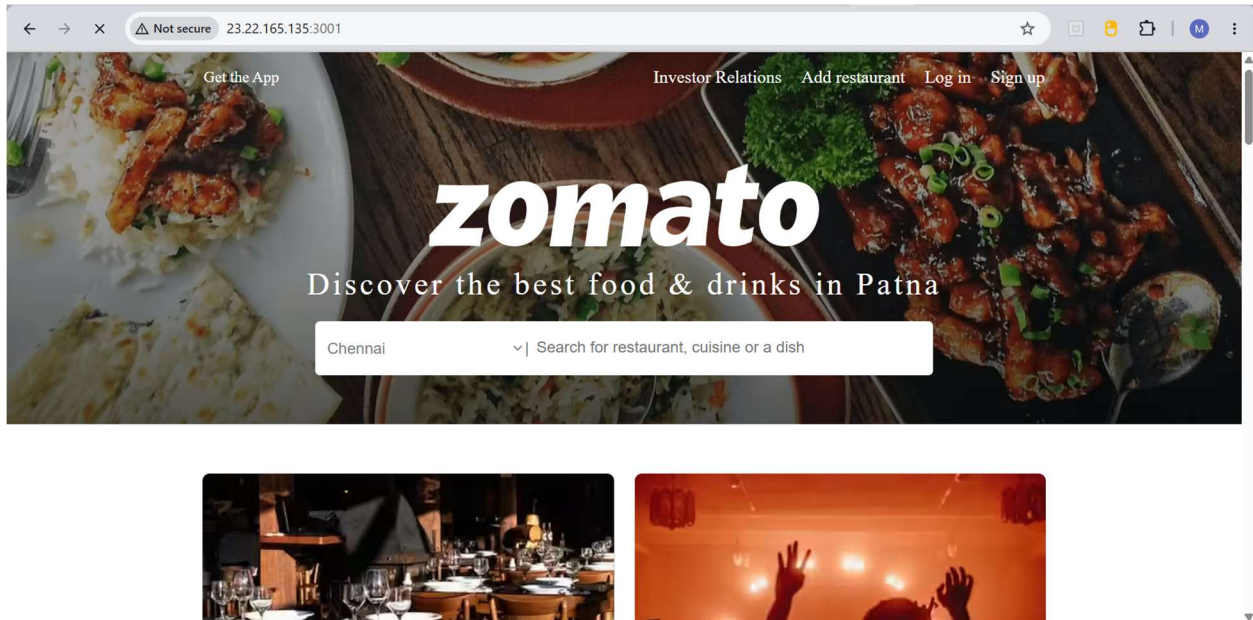
docker run -d --name node-container -p 3001:3000 node-app:v1

```

root@ip-172-31-26-239:/home/ubuntu/docker-tasks/NodeJs_based_source# docker run -d --name node-container -p 3001:3000 node-app:v1
960fb5ab25c3d61e3456a88cf6511a2538c04eb63210f0be60f644a8abdbc29d
root@ip-172-31-26-239:/home/ubuntu/docker-tasks/NodeJs_based_source#

```

→ We are able to access the browser



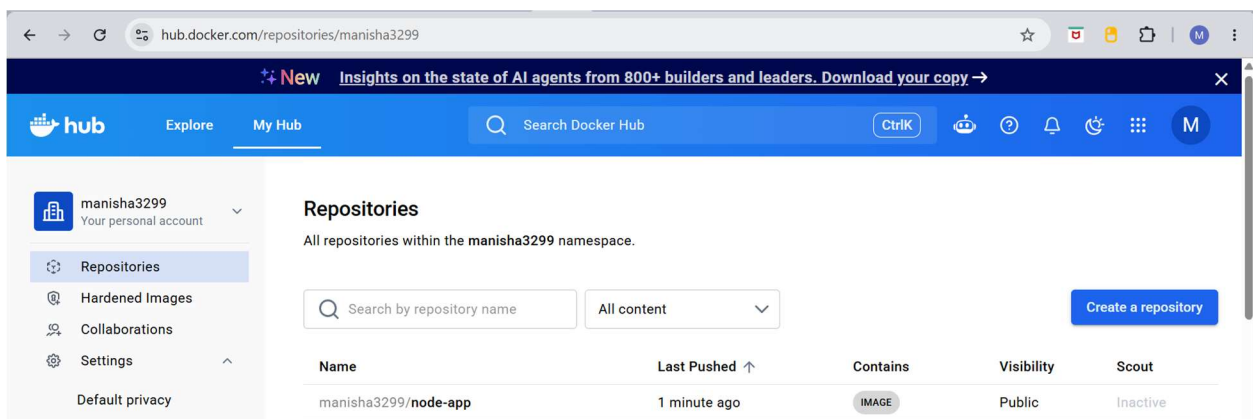
→ Now let's push to Docker Hub

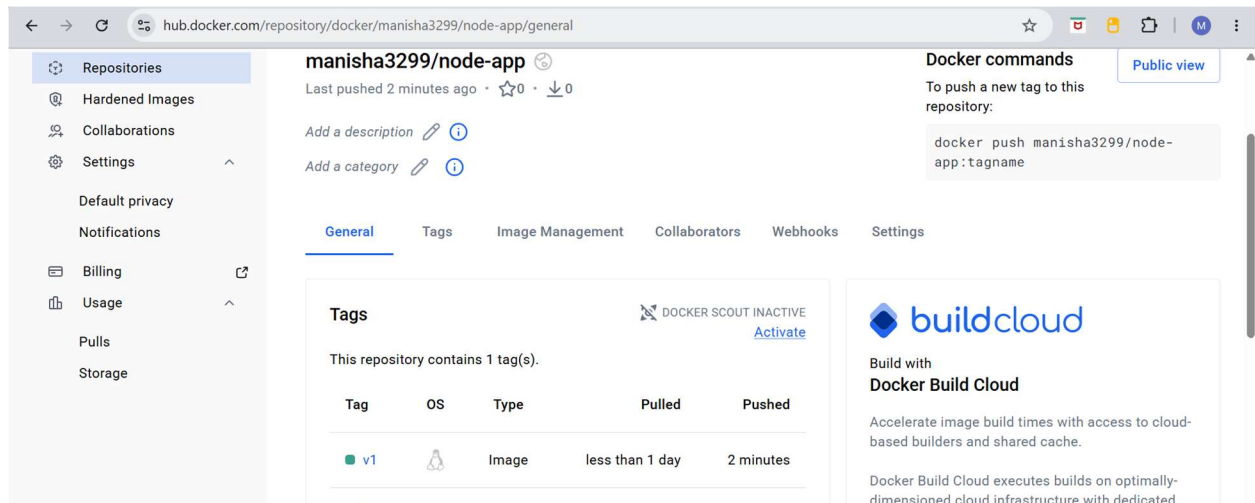
docker tag node-app:v1 manisha3299/node-app:v1

docker push manisha3299/node-app:v1

```
root@ip-172-31-26-239:/home/ubuntu/docker-tasks/NodeJs_based_source# docker tag node-app:v1 manisha3299/node-app:v1
root@ip-172-31-26-239:/home/ubuntu/docker-tasks/NodeJs_based_source# docker push manisha3299/node-app:v1
The push refers to repository [docker.io/manisha3299/node-app]
dd71dde834b5: Mounted from library/node
4b41e5ae0521: Pushed
36bde7c5c114: Pushed
a7e973accd28: Pushed
9bac93d1f13f: Pushed
1e5a4c89cee5: Mounted from library/node
f18232174bc9: Mounted from library/node
25ff2da83641: Mounted from library/node
v1: digest: sha256:68ab2a1ea75c7b98f68b14871f101e8489cba880244356faed5a16d0dc5de098 size: 2062
root@ip-172-31-26-239:/home/ubuntu/docker-tasks/NodeJs_based_source#
```

Image pushed successfully





→ Write a docker-compose file to set up WordPress with a MySQL database

First lets create a docker-compose.yml file

```
root@ip-172-31-26-239: /home/ubuntu
version: '3'
services:
  db:
    image: mysql:5.7
    volumes:
      - db_data:/var/lib/mysql
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: somewordpress
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wordpress
      MYSQL_PASSWORD: wordpress
  wordpress:
    depends_on:
      - db
    image: wordpress:latest
    ports:
      - "8000:80"
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wordpress
      WORDPRESS_DB_PASSWORD: wordpress
      WORDPRESS_DB_NAME: wordpress
volumes:
  db_data:
```

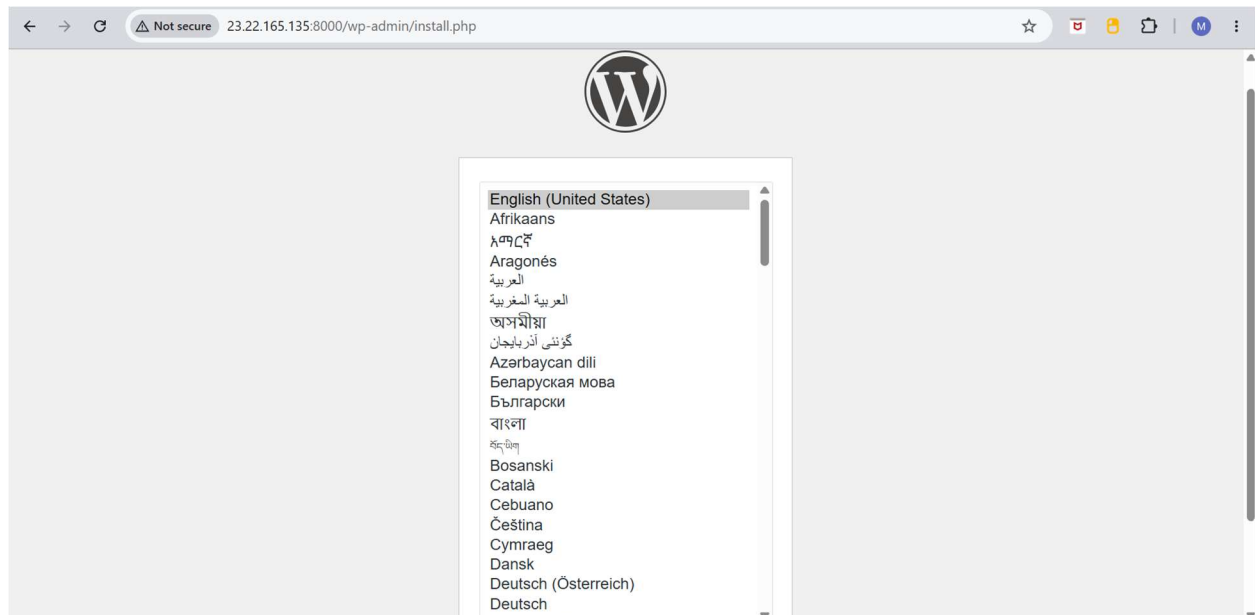
docker compose up


```
root@ip-172-31-26-239: /home/ubuntu

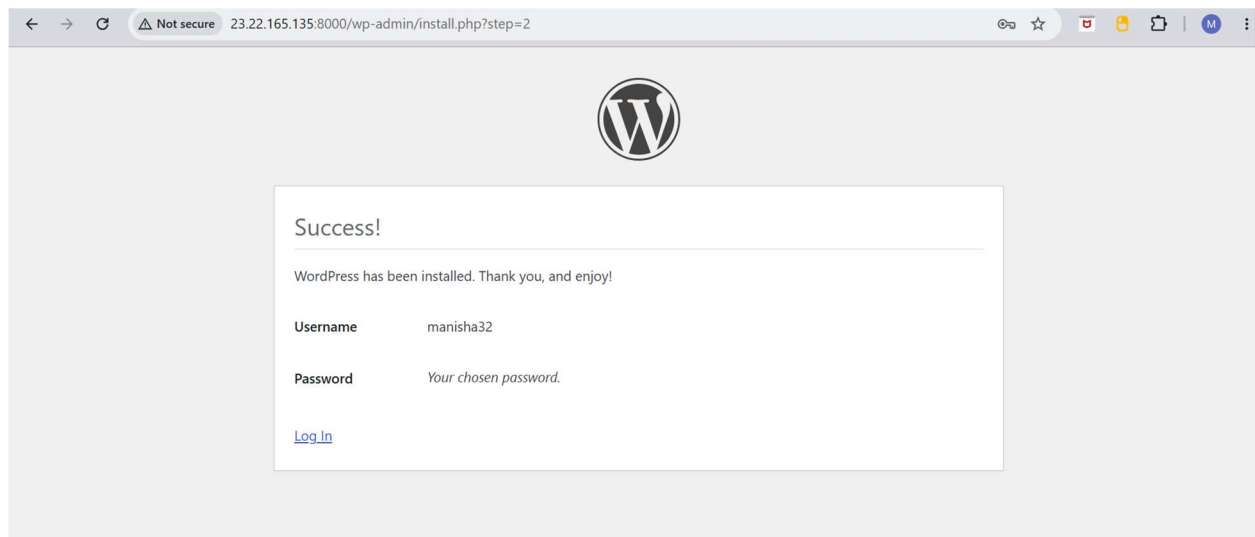
[+] Running 4/4
✔ Network ubuntu_default          Created          0.0s
✔ Volume ubuntu_db_data           Created          0.0s
✔ Container ubuntu-db-1           Created          1.2s
✔ Container ubuntu-wordpress-1    Created          0.1s
Attaching to db-1, wordpress-1
db-1 | 2026-06-06 15:14:25+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 5.7.44-1.e
17 started.
wordpress-1 | WordPress not found in /var/www/html - copying now...
wordpress-1 | Complete! WordPress has been successfully copied to /var/www/html
wordpress-1 | No 'wp-config.php' found in /var/www/html, but 'WORDPRESS_...' variables supplied; co
pying 'wp-config-docker.php' (WORDPRESS_DB_HOST WORDPRESS_DB_NAME WORDPRESS_DB_PASSWORD WORDPRESS_DB
USER)
db-1 | 2026-06-06 15:14:25+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
db-1 | 2026-06-06 15:14:25+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 5.7
.44-1.e17 started.
db-1 | 2026-06-06 15:14:25+00:00 [Note] [Entrypoint]: Initializing database files
db-1 | 2026-06-06T15:14:25.508886Z 0 [Warning] TIMESTAMP with implicit DEFAULT value is depr
ecated. Please use --explicit_defaults_for_timestamp server option (see documentation for more detail
s).
wordpress-1 | AH00558: apache2: Could not reliably determine the server's fully qualified domain na
me, using 172.18.0.3. Set the 'ServerName' directive globally to suppress this message
db-1 | 2026-06-06T15:14:26.418890Z 0 [Warning] InnoDB: New log files created, LSN=45790
wordpress-1 | AH00558: apache2: Could not reliably determine the server's fully qualified domain na
me, using 172.18.0.3. Set the 'ServerName' directive globally to suppress this message
db-1 | 2026-06-06T15:14:26.638880Z 0 [Warning] InnoDB: Creating foreign key constraint system
tables.
db-1 | 2026-06-06T15:14:26.853101Z 0 [Warning] No existing UUID has been found, so we assume
that this is the first time that this server has been started. Generating a new UUID: 68661f84-61ba
-11f1-9393-7ab70bb65024.
db-1 | 2026-06-06T15:14:26.884394Z 0 [Warning] Gtid table is not ready to be used. Table 'my
sql.gtid_executed' cannot be opened.
db-1 | 2026-06-06T15:14:27.076936Z 0 [Warning] A deprecated TLS version TLSv1 is enabled. Pl

db-1 | 2026-06-06T15:14:34.449746Z 0 [Note] InnoDB: 96 redo rollback segment(s) found. 96 re
do rollback segment(s) are active.
db-1 | 2026-06-06T15:14:34.449762Z 0 [Note] InnoDB: 32 non-redo rollback segment(s) are acti
ve.
db-1 | 2026-06-06T15:14:34.450053Z 0 [Note] InnoDB: Waiting for purge to start
db-1 | 2026-06-06T15:14:34.500276Z 0 [Note] InnoDB: 5.7.44 started; log sequence number 1221
9253
db-1 | 2026-06-06T15:14:34.500511Z 0 [Note] InnoDB: Loading buffer pool(s) from /var/lib/mys
ql/ib_buffer_pool
db-1 | 2026-06-06T15:14:34.500628Z 0 [Note] Plugin 'FEDERATED' is disabled.
db-1 | 2026-06-06T15:14:34.504828Z 0 [Note] InnoDB: Buffer pool(s) load completed at 260606
15:14:34
db-1 | 2026-06-06T15:14:34.506428Z 0 [Note] Found ca.pem, server-cert.pem and server-key.pem
in data directory. Trying to enable SSL support using them.
db-1 | 2026-06-06T15:14:34.506437Z 0 [Note] Skipping generation of SSL certificates as certi
ficate files are present in data directory.
db-1 | 2026-06-06T15:14:34.506441Z 0 [Warning] A deprecated TLS version TLSv1 is enabled. Pl
ease use TLSv1.2 or higher.
db-1 | 2026-06-06T15:14:34.506443Z 0 [Warning] A deprecated TLS version TLSv1.1 is enabled.
Please use TLSv1.2 or higher.
db-1 | 2026-06-06T15:14:34.507143Z 0 [Warning] CA certificate ca.pem is self signed.
db-1 | 2026-06-06T15:14:34.507309Z 0 [Note] Skipping generation of RSA key pair as key files
are present in data directory.
db-1 | 2026-06-06T15:14:34.507774Z 0 [Note] Server hostname (bind-address): '*'; port: 3306
db-1 | 2026-06-06T15:14:34.508796Z 0 [Note] IPv6 is available.
db-1 | 2026-06-06T15:14:34.509528Z 0 [Note] - '::' resolves to '::';
db-1 | 2026-06-06T15:14:34.509616Z 0 [Note] Server socket created on IP: '::'.
db-1 | 2026-06-06T15:14:34.511930Z 0 [Warning] Insecure configuration for --pid-file: Locati
on '/var/run/mysqld' in the path is accessible to all OS users. Consider choosing a different direct
ory.
db-1 | 2026-06-06T15:14:34.517032Z 0 [Note] Event Scheduler: Loaded 0 events
db-1 | 2026-06-06T15:14:34.517407Z 0 [Note] mysqld: ready for connections.
db-1 | Version: '5.7.44' socket: '/var/run/mysqld/mysqld.sock' port: 3306 MySQL Community
Server (GPL)
```

→ Now we are able to access the browser



→ Successfully installed wordpress



→ Now lets login to wordpress

← → ↻ ⚠ Not secure 23.22.165.135:8000/wp-login.php 🔍 ☆ 📄 📁 📌 M



Username or Email Address

Password

☐ Remember Me

[Lost your password?](#)

[← Go to](#)

← → ↻ ⚠ Not secure 23.22.165.135:8000/wp-admin/ 🔍 ☆ 📄 📁 📌 M



23.22.165.135:8000 🔍 Ctrl+K 0 + New

Howdy, manisha32  

Dashboard

Home

Updates

Posts

Media

Pages

Comments

Appearance

Plugins

Users

Tools

Settings

Collapse Menu

Dashboard

Welcome to WordPress!

[Learn more about the 7.0 version.](#)

 Author rich content with blocks and patterns

Block patterns are pre-configured block layouts. Use them to get inspired or create new pages in a flash.

[Add a new page](#)

 Customize your entire site with block themes

Design everything on your site — from the header down to the footer, all using blocks and patterns.

[Open site editor](#)

 Switch up your site's look & feel with Styles

Tweak your site, or give it a whole new look! Get creative — how about a new color palette or font?

[Edit styles](#)


```
root@ip-172-31-26-239: /home/ubuntu
Hit_Locale=user HTTP/1.1" 200 3085 "http://23.22.165.135:8000/wp-admin/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36"
wordpress-1 | 49.43.230.167 - - [06/Jun/2026:15:22:10 +0000] "GET /wp-admin/admin-ajax.php?action=wp-compression-test&test=1&ajax_nonce=6a06726d11&1780759330579 HTTP/1.1" 200 1188 "http://23.22.165.135:8000/wp-admin/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36"
wordpress-1 | 49.43.230.167 - - [06/Jun/2026:15:22:10 +0000] "GET /wp-includes/js/thickbox/loadingAnimation.gif HTTP/1.1" 200 15525 "http://23.22.165.135:8000/wp-admin/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36"
wordpress-1 | 49.43.230.167 - - [06/Jun/2026:15:22:10 +0000] "POST /wp-admin/admin-ajax.php HTTP/1.1" 200 634 "http://23.22.165.135:8000/wp-admin/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36"
wordpress-1 | 49.43.230.167 - - [06/Jun/2026:15:22:11 +0000] "GET /wp-admin/admin-ajax.php?action=wp-compression-test&test=no&ajax_nonce=6a06726d11&1780759330961 HTTP/1.1" 200 539 "http://23.22.165.135:8000/wp-admin/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36"
wordpress-1 | 49.43.230.167 - - [06/Jun/2026:15:22:10 +0000] "GET /wp-admin/admin-ajax.php?action=dashboard-widgets&widget=dashboard_primary&pagenow=dashboard HTTP/1.1" 200 884 "http://23.22.165.135:8000/wp-admin/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36"
wordpress-1 | 127.0.0.1 - - [06/Jun/2026:15:22:18 +0000] "OPTIONS * HTTP/1.0" 200 126 "-" "Apache/2.4.67 (Debian) PHP/8.3.31 (internal dummy connection)"
wordpress-1 | 23.22.165.135 - - [06/Jun/2026:15:23:11 +0000] "POST /wp-cron.php?doing_wp_cron=1780759391.5045540332794189453125 HTTP/1.1" 200 285 "-" "WordPress/7.0; http://23.22.165.135:8000"
wordpress-1 | 49.43.230.167 - - [06/Jun/2026:15:23:11 +0000] "POST /wp-admin/admin-ajax.php HTTP/1.1" 200 581 "http://23.22.165.135:8000/wp-admin/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36"
wordpress-1 | 49.43.230.167 - - [06/Jun/2026:15:24:03 +0000] "-" 408 0 "-" "-"
wordpress-1 | 49.43.230.167 - - [06/Jun/2026:15:24:12 +0000] "POST /wp-admin/admin-ajax.php HTTP/1.1" 200 581 "http://23.22.165.135:8000/wp-admin/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36"
Enable Watch
```

If we check in another cli wordpress and my sql are running

```
root@ip-172-31-26-239:/home/ubuntu# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        NAME
7ded59a04ce9   wordpress:latest "docker-entrypoint.s..." 9 minutes ago   Up 9 minutes   ubuntu-wordpress-1
0.0.0.0:8000->80/tcp, [::]:8000->80/tcp
7a75e6b9069c   mysql:5.7      "docker-entrypoint.s..." 9 minutes ago   Up 9 minutes   ubuntu-db-1
3306/tcp, 33060/tcp
```

Docker compose ps → shows containers running in docker compose

```
root@ip-172-31-26-239:/home/ubuntu# docker compose ps
WARN[0000] /home/ubuntu/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME          IMAGE          COMMAND                  SERVICE   CREATED        STATUS
ubuntu-db-1   mysql:5.7      "docker-entrypoint.s..." db        11 minutes ago Up 11 minutes
3306/tcp, 33060/tcp
ubuntu-wordpress-1 wordpress:latest "docker-entrypoint.s..." wordpress 11 minutes ago Up 11 minutes
0.0.0.0:8000->80/tcp, [::]:8000->80/tcp
root@ip-172-31-26-239:/home/ubuntu#
```